

1

一

二

三

四

五

六

七

八

九

十

1

十一 附录

附录 1：支撑材料，文件列表附录 2：补充表格，图片附录 3：代码

附录 1：支撑材料文件列表

附录 1：支撑材料文件列表	
文件列表名	
问题一	混合模型（R 语言代码） 附件
问题二	C(2)(权重计算).py C(2)(生存函数).py C(2) 测试误差.py C(2)(权重计算).py C(2) 聚类.py 不同组的累积达标曲线、效用函数曲线及最优时点标注图 不同组的误差模拟（时点分布、效用值分布及风险分布）图
问题三	C(3)preprocessing.py Q3 数据预处理.xlsx 聚类 (C(3) 聚类.py+ 图片) C(3)final (C(3)final+ 相关运行结果 + 图片)
问题四	数据处理.py 点二列相关系数 __smote__ 可视化 __final.py

附录 2：补充表格，图片

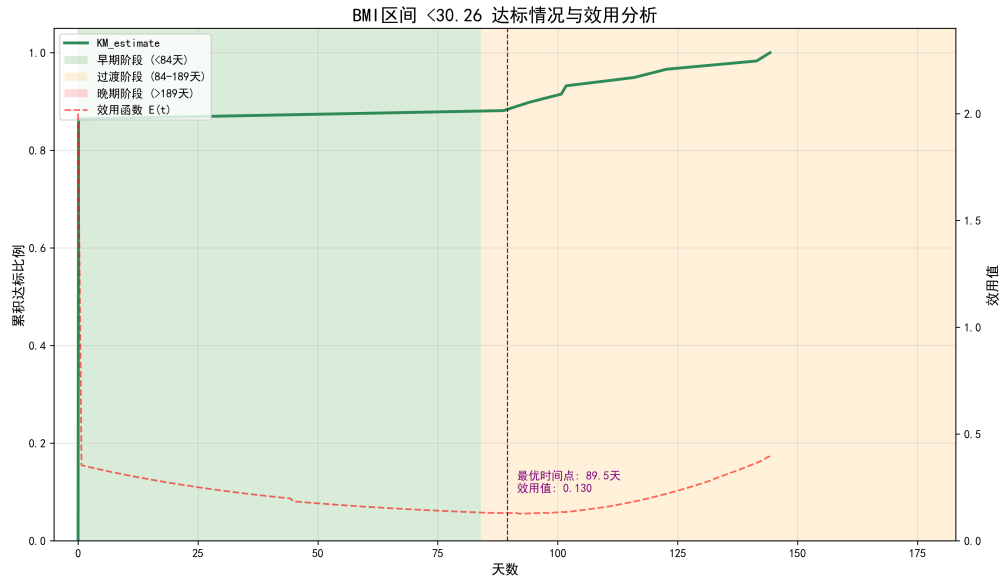


图 1: 累积达标曲线（小于 30.26 组）

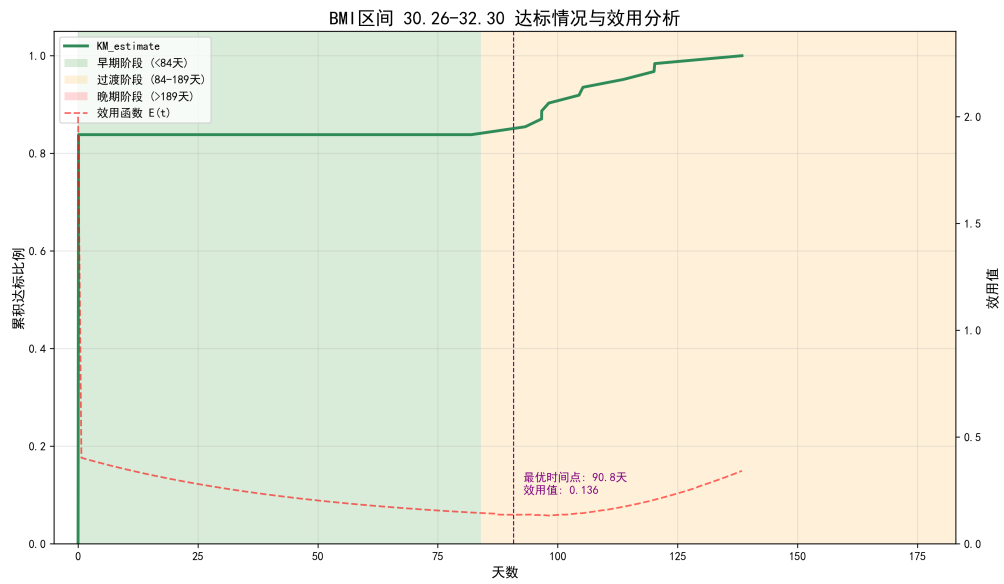


图 2: 累积达标曲线（30.26-32.30 组）

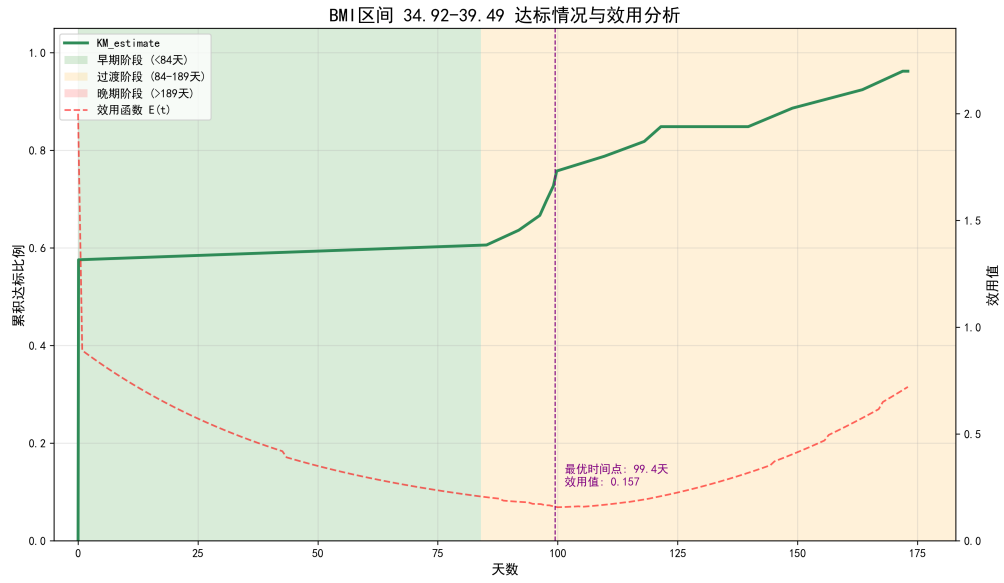


图 3: 累积达标曲线 (34.92-39.49 组)

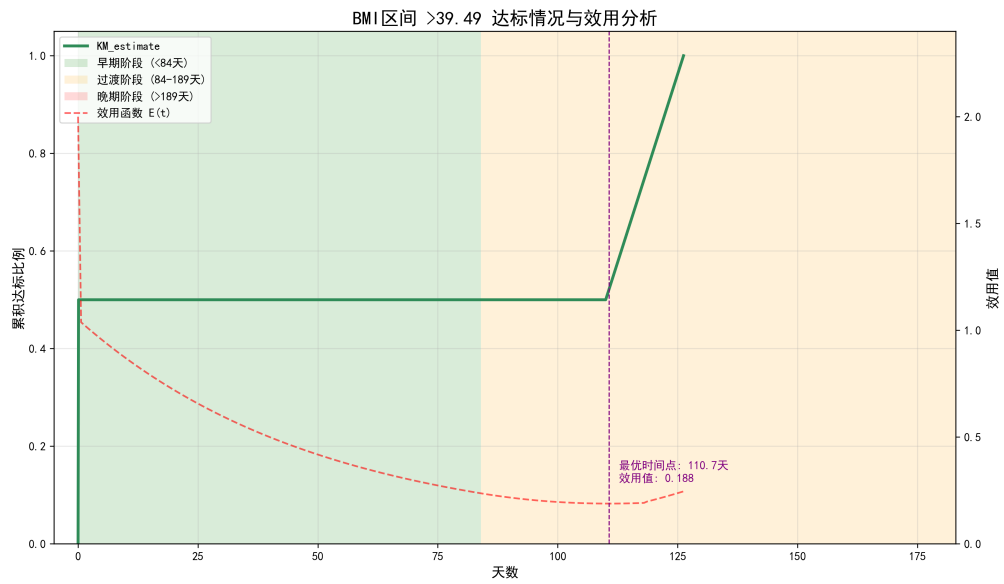


图 4: 累积达标曲线 (大于 39.49 组)

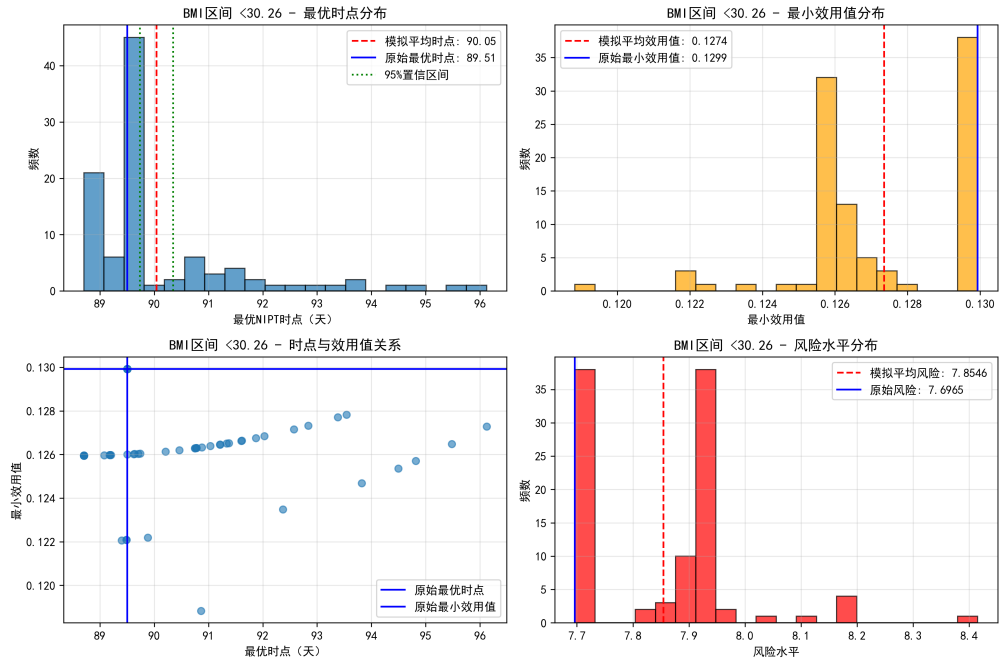


图 5: 误差模拟 (小于 30.26 组)

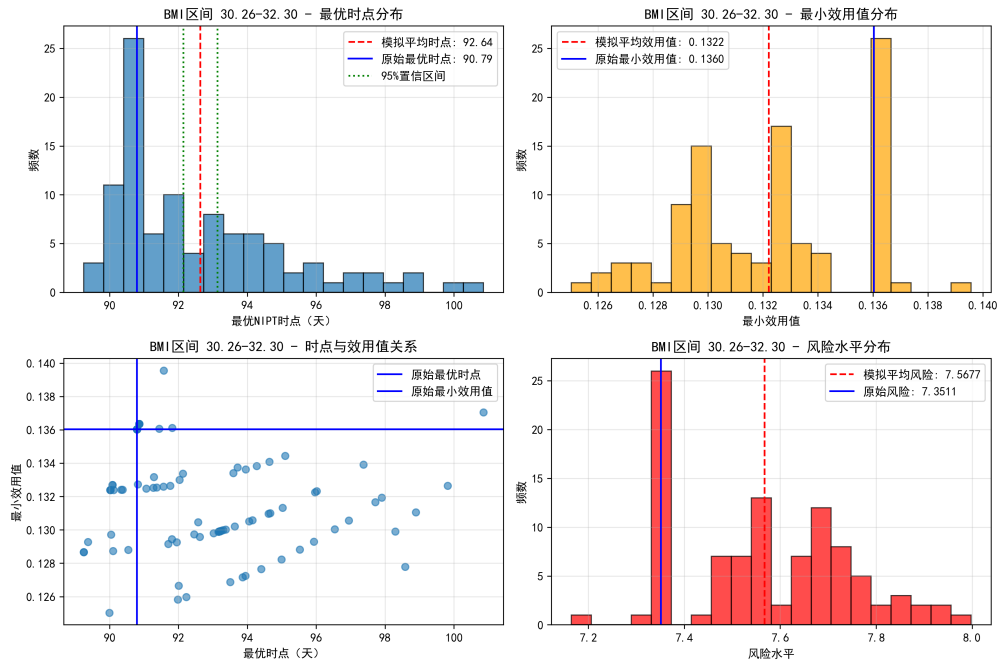


图 6: 误差模拟 (30.26-32.30 组)

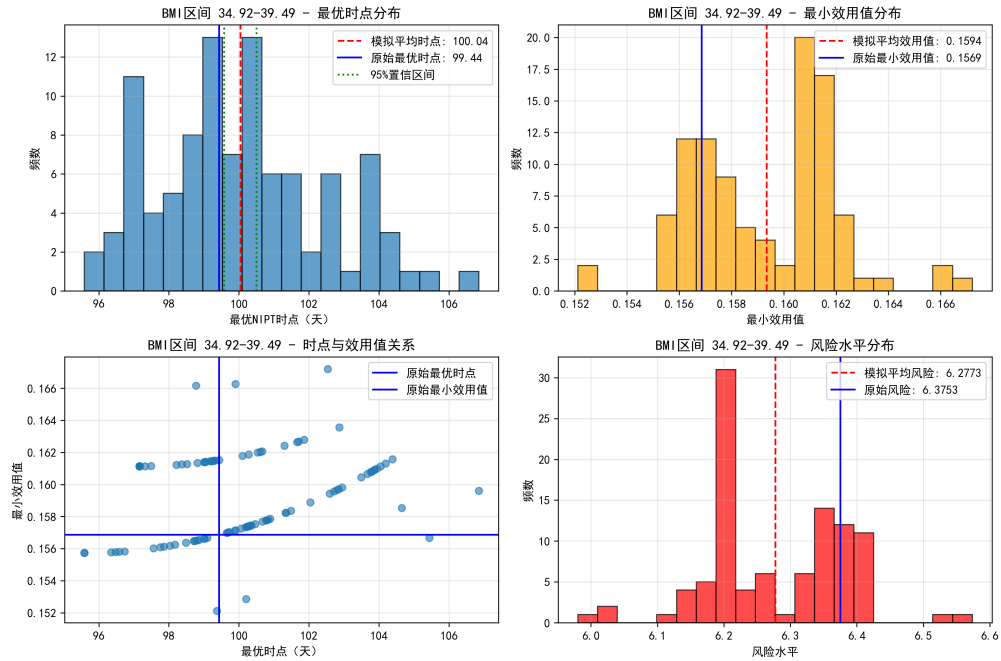


图 7: 误差模拟 (34.92-39.49 组)

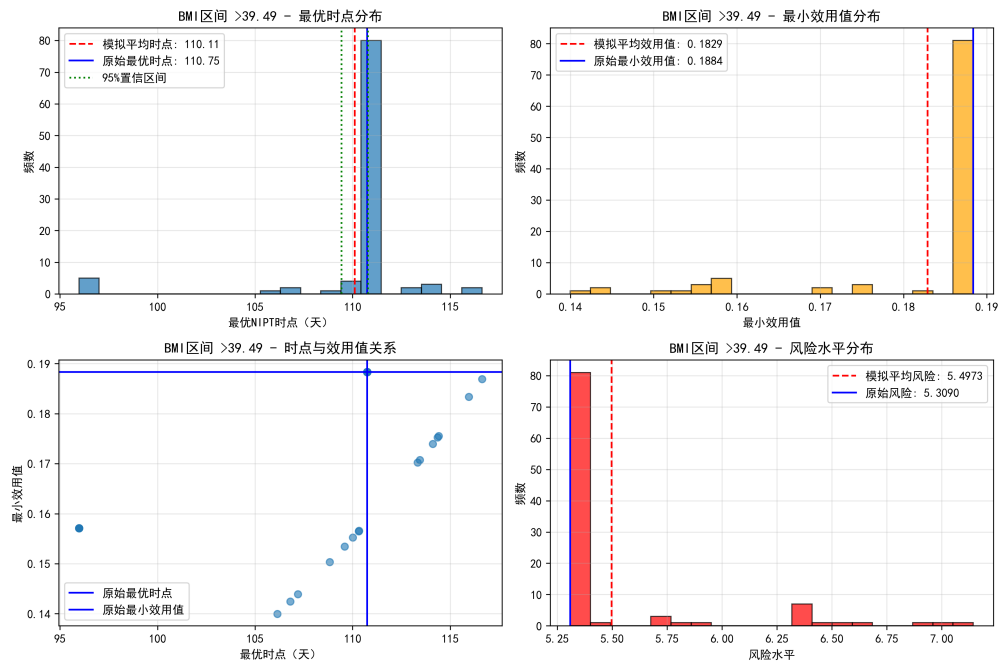


图 8: 误差模拟 (大于 39.49 组)

Listing 1: 终版非线性混合效应.R

```

1 # 加载必要的包
2 library(readxl)
3 library(nlme)

```

```

4 library(ggplot2)
5 library(lme4)
6 library(MuMIn)
7 library(mgcv) # 用于GAMM模型
8 library(merTools)
9 library(sjPlot)
10 library(gridExtra)
11 library(itsadug) # 用于GAM模型可视化
12
13 # 读取数据
14 data_path <- "D:/math_modeling/25_C/mathmodeling_2024/python_code/Q1/C(1)预处理.xlsx"
15 data <- read_excel(data_path)
16
17 # 查看数据结构
18 str(data)
19 head(data)
20
21 # 重命名列名以便使用
22 colnames(data) <- c("Subject", "BMI", "Y_concentration", "Gestational_days")
23
24 # 数据预处理
25 data$Subject <- as.factor(data$Subject)
26 data$BMI <- as.numeric(data$BMI)
27 data$Y_concentration <- as.numeric(data$Y_concentration)
28 data$Gestational_days <- as.numeric(data$Gestational_days)
29
30 # 探索性数据分析
31 exploratory_plot <- ggplot(data, aes(x = Gestational_days, y = Y_concentration, color = Subject)) +
32   geom_point() +
33   geom_smooth(method = "lm", se = FALSE) +
34   theme_minimal() +
35   labs(title = "Y染色体浓度随孕周变化", x = "孕周(天)", y = "Y染色体浓度")
36
37 print(exploratory_plot)
38 ggsave("exploratory_plot.png", width = 10, height = 6, dpi = 300)
39
40 # 1. 线性混合模型 (LMM)
41 cat("拟合线性混合模型...\n")
42 lmm_model <- lmer(Y_concentration ~ BMI + Gestational_days + (1|Subject), data = data)
43 cat("线性混合模型拟合完成\n")
44 print(summary(lmm_model))
45
46 # 2. 指数模型
47 cat("拟合指数模型...\n")
48 exp_model_formula <- function(bmi, days, a, b, c) {
49   a * exp(b * bmi + c * days)
50 }
51
52 tryCatch({
53   exp_nlme <- nlme(Y_concentration ~ exp_model_formula(BMI, Gestational_days, a, b, c),
54     data = data,
55     fixed = a + b + c ~ 1,
56     random = a ~ 1 | Subject,
57     start = list(a = 0.05, b = 0.001, c = 0.001),
58     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))

```

```

59   cat("指数模型拟合成功\n")
60   print(summary(exp_nlme))
61 }, error = function(e) {
62   message("指数模型拟合失败: ", e$message)
63   exp_nlme <- NULL
64 })
65
66 # 3. Logistic生长模型
67 cat("拟合Logistic模型...\n")
68 logistic_model_formula <- function(bmi, days, a, b, c, d) {
69   a / (1 + exp(-(b * bmi + c * days - d)))
70 }
71
72 tryCatch({
73   logistic_nlme <- nlme(Y_concentration ~ logistic_model_formula(BMI, Gestational_days, a, b, c, d),
74     data = data,
75     fixed = a + b + c + d ~ 1,
76     random = a ~ 1 | Subject,
77     start = list(a = 0.1, b = 0.01, c = 0.01, d = 50),
78     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))
79   cat("Logistic模型拟合成功\n")
80   print(summary(logistic_nlme))
81 }, error = function(e) {
82   message("Logistic模型拟合失败: ", e$message)
83   logistic_nlme <- NULL
84 })
85
86 # 4. Gompertz生长模型
87 cat("拟合Gompertz模型...\n")
88 gompertz_model_formula <- function(bmi, days, a, b, c, d) {
89   a * exp(-exp(-(b * bmi + c * days - d)))
90 }
91
92 tryCatch({
93   gompertz_nlme <- nlme(Y_concentration ~ gompertz_model_formula(BMI, Gestational_days, a, b, c, d),
94     data = data,
95     fixed = a + b + c + d ~ 1,
96     random = a ~ 1 | Subject,
97     start = list(a = 0.1, b = 0.01, c = 0.01, d = 50),
98     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))
99   cat("Gompertz模型拟合成功\n")
100   print(summary(gompertz_nlme))
101 }, error = function(e) {
102   message("Gompertz模型拟合失败: ", e$message)
103   gompertz_nlme <- NULL
104 })
105
106 # 5. 广义加性混合模型 (GAMM)
107 cat("拟合广义加性混合模型...\n")
108 tryCatch({
109   # 使用bam函数拟合GAMM, 适用于大型数据集
110   gamm_model <- bam(Y_concentration ~ s(BMI) + s(Gestational_days) + s(Subject, bs = "re"),
111     data = data,
112     method = "REML")
113   cat("GAMM模型拟合成功\n")

```

```

114 print(summary(gamm_model))
115
116 # 绘制平滑项图
117 png("gamm_smooth_terms.png", width = 10, height = 6, units = "in", res = 300)
118 par(mfrow = c(1, 2))
119 plot(gamm_model, select = 1, main = "BMI平滑项")
120 plot(gamm_model, select = 2, main = "孕周平滑项")
121 dev.off()
122 }, error = function(e) {
123   message("GAMM模型拟合失败: ", e$message)
124   gamm_model <- NULL
125 })
126
127 # 模型比较
128 cat("开始模型比较...\n")
129 model_comparison <- data.frame(
130   Model = "线性混合模型",
131   AIC = AIC(lmm_model),
132   BIC = BIC(lmm_model),
133   LogLik = as.numeric(logLik(lmm_model))
134 )
135
136 # 添加非线性模型 (如果成功拟合)
137 if(exists("exp_nlme") && !is.null(exp_nlme)) {
138   model_comparison <- rbind(model_comparison,
139     data.frame(Model = "指数模型",
140       AIC = AIC(exp_nlme),
141       BIC = BIC(exp_nlme),
142       LogLik = logLik(exp_nlme)))
143 }
144
145 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
146   model_comparison <- rbind(model_comparison,
147     data.frame(Model = "Logistic模型",
148       AIC = AIC(logistic_nlme),
149       BIC = BIC(logistic_nlme),
150       LogLik = logLik(logistic_nlme)))
151 }
152
153 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
154   model_comparison <- rbind(model_comparison,
155     data.frame(Model = "Gompertz模型",
156       AIC = AIC(gompertz_nlme),
157       BIC = BIC(gompertz_nlme),
158       LogLik = logLik(gompertz_nlme)))
159 }
160
161 if(exists("gamm_model") && !is.null(gamm_model)) {
162   model_comparison <- rbind(model_comparison,
163     data.frame(Model = "GAMM模型",
164       AIC = AIC(gamm_model),
165       BIC = BIC(gamm_model),
166       LogLik = logLik(gamm_model)))
167 }
168

```

```

169 # 显示模型比较结果
170 print("模型比较结果:")
171 print(model_comparision)
172
173 # 可视化拟合结果
174 # 创建预测函数
175 predict_nlme <- function(model, newdata) {
176   if(inherits(model, "nlme")) {
177     return(predict(model, newdata = newdata, level = 0))
178   } else if(inherits(model, "lmerMod")) {
179     return(predict(model, newdata = newdata, re.form = NA))
180   } else if(inherits(model, "bam")) {
181     return(predict(model, newdata = newdata, exclude = "s(Subject)"))
182   } else {
183     return(rep(NA, nrow(newdata)))
184   }
185 }
186
187 # 生成预测数据 - 修复: 使用平均BMI值
188 pred_data <- data.frame(
189   BMI = rep(mean(data$BMI), 100), # 使用平均BMI值
190   Gestational_days = seq(min(data$Gestational_days), max(data$Gestational_days), length.out = 100),
191   Subject = rep(levels(data$Subject)[1], 100) # 使用第一个受试者
192 )
193
194 # 为每个模型生成预测
195 pred_data$lmm <- predict_nlme(lmm_model, pred_data)
196
197 if(exists("exp_nlme") && !is.null(exp_nlme)) {
198   pred_data$exp <- predict_nlme(exp_nlme, pred_data)
199 }
200
201 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
202   pred_data$logistic <- predict_nlme(logistic_nlme, pred_data)
203 }
204
205 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
206   pred_data$gompertz <- predict_nlme(gompertz_nlme, pred_data)
207 }
208
209 if(exists("gamm_model") && !is.null(gamm_model)) {
210   pred_data$gamm <- predict_nlme(gamm_model, pred_data)
211 }
212
213 # 创建每个模型的拟合图
214 plots <- list()
215
216 # 线性混合模型
217 plots[[1]] <- ggplot() +
218   geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha = 0.6)
219   +
220   geom_line(data = pred_data, aes(x = Gestational_days, y = lmm, color = "总体拟合"), size = 1) +
221   theme_minimal() +
222   labs(title = "线性混合模型拟合", x = "孕周(天)", y = "Y染色体浓度")

```



```

223 # 指数模型
224 if(exists("exp_nlme") && !is.null(exp_nlme)) {
225   plots[[2]] <- ggplot() +
226     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
227       0.6) +
228     geom_line(data = pred_data, aes(x = Gestational_days, y = exp, color = "总体拟合"), size = 1) +
229     theme_minimal() +
230     labs(title = "指数模型拟合", x = "孕周(天)", y = "Y染色体浓度")
231 }
232
233 # Logistic模型
234 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
235   plots[[3]] <- ggplot() +
236     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
237       0.6) +
238     geom_line(data = pred_data, aes(x = Gestational_days, y = logistic, color = "总体拟合"), size = 1) +
239     theme_minimal() +
240     labs(title = "Logistic模型拟合", x = "孕周(天)", y = "Y染色体浓度")
241 }
242
243 # Gompertz模型
244 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
245   plots[[4]] <- ggplot() +
246     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
247       0.6) +
248     geom_line(data = pred_data, aes(x = Gestational_days, y = gompertz, color = "总体拟合"), size = 1) +
249     theme_minimal() +
250     labs(title = "Gompertz模型拟合", x = "孕周(天)", y = "Y染色体浓度")
251 }
252
253 # GAMM模型
254 if(exists("gamm_model") && !is.null(gamm_model)) {
255   plots[[5]] <- ggplot() +
256     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
257       0.6) +
258     geom_line(data = pred_data, aes(x = Gestational_days, y = gamm, color = "总体拟合"), size = 1) +
259     theme_minimal() +
260     labs(title = "GAMM模型拟合", x = "孕周(天)", y = "Y染色体浓度")
261 }
262
263 # 保存所有图表
264 for(i in seq_along(plots)) {
265   ggsave(paste0("model_fit_", i, ".png"), plots[[i]], width = 10, height = 6, dpi = 300)
266 }
267
268 # 创建模型比较图
269 model_comp_plot <- ggplot(model_comparison, aes(x = reorder(Model, AIC), y = AIC, fill = Model)) +
270   geom_bar(stat = "identity") +
271   theme_minimal() +
272   labs(title = "模型比较 - AIC值", x = "模型", y = "AIC值") +
273   theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
274   scale_fill_brewer(palette = "Set2")
275
276 ggsave("model_comparison_aic.png", model_comp_plot, width = 10, height = 6, dpi = 300)
277

```

```

274 # 残差分析
275 residual_plots <- list()
276
277 # 线性混合模型残差
278 residual_plots[[1]] <- ggplot(data, aes(x = fitted(lmm_model), y = resid(lmm_model))) +
279   geom_point(alpha = 0.6) +
280   geom_hline(yintercept = 0, linetype = "dashed") +
281   theme_minimal() +
282   labs(title = "线性混合模型残差图", x = "拟合值", y = "残差")
283
284 # 指数模型残差
285 if(exists("exp_nlme") && !is.null(exp_nlme)) {
286   residual_plots[[2]] <- ggplot(data, aes(x = fitted(exp_nlme), y = resid(exp_nlme))) +
287     geom_point(alpha = 0.6) +
288     geom_hline(yintercept = 0, linetype = "dashed") +
289     theme_minimal() +
290     labs(title = "指数模型残差图", x = "拟合值", y = "残差")
291 }
292
293 # GAMM模型残差
294 if(exists("gamm_model") && !is.null(gamm_model)) {
295   residual_plots[[3]] <- ggplot(data, aes(x = fitted(gamm_model), y = resid(gamm_model))) +
296     geom_point(alpha = 0.6) +
297     geom_hline(yintercept = 0, linetype = "dashed") +
298     theme_minimal() +
299     labs(title = "GAMM模型残差图", x = "拟合值", y = "残差")
300 }
301
302 # 保存残差图
303 for(i in seq_along(residual_plots)) {
304   ggsave(paste0("residual_plot_", i, ".png"), residual_plots[[i]], width = 10, height = 6, dpi = 300)
305 }
306
307 # 输出模型摘要
308 sink("model_summaries.txt")
309 cat("线性混合模型摘要:\n")
310 print(summary(lmm_model))
311
312 if(exists("exp_nlme") && !is.null(exp_nlme)) {
313   cat("\n指数模型摘要:\n")
314   print(summary(exp_nlme))
315 }
316
317 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
318   cat("\nLogistic模型摘要:\n")
319   print(summary(logistic_nlme))
320 }
321
322 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
323   cat("\nGompertz模型摘要:\n")
324   print(summary(gompertz_nlme))
325 }
326
327 if(exists("gamm_model") && !is.null(gamm_model)) {
328   cat("\nGAMM模型摘要:\n")

```

```

329     print(summary(gamm_model))
330 }
331
332 cat("\n模型比较:\n")
333 print(model_comparison)
334 sink()
335
336 # 显示最佳模型
337 if(nrow(model_comparison) > 0) {
338     best_model_idx <- which.min(model_comparison$AIC)
339     cat("根据AIC准则, 最佳模型是:", model_comparison$Model[best_model_idx], "\n")
340
341     # 绘制最佳模型拟合图
342     if(model_comparison$Model[best_model_idx] == "线性混合模型") {
343         best_plot <- plots[[1]]
344     } else if(model_comparison$Model[best_model_idx] == "指数模型") {
345         best_plot <- plots[[2]]
346     } else if(model_comparison$Model[best_model_idx] == "Logistic模型") {
347         best_plot <- plots[[3]]
348     } else if(model_comparison$Model[best_model_idx] == "Gompertz模型") {
349         best_plot <- plots[[4]]
350     } else if(model_comparison$Model[best_model_idx] == "GAMM模型") {
351         best_plot <- plots[[5]]
352     }
353
354     ggsave("best_model_fit.png", best_plot, width = 10, height = 6, dpi = 300)
355 }
356
357 # 保存工作空间
358 save.image("mixed_model_analysis.RData")
359
360 cat("分析完成! 所有结果已保存到当前工作目录。 \n")

```

Listing 2: C(2)(生存函数).py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from lifelines import KaplanMeierFitter
5  from scipy.optimize import minimize_scalar
6
7  # 设置中文字体支持
8  plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
9  plt.rcParams["axes.unicode_minus"] = False
10
11
12 def analyze_with_dynamic_utility():
13     # 定义风险函数
14     def get_rearly(t):
15         return 2.0 * np.exp(-t / 50)
16
17     def get_rlate(t):
18         if t < 84:
19             return 0.1
20         elif t <= 189:

```

```

21         normalized_t = (t - 84) / (189 - 84)
22         return 0.1 + 0.9 * (normalized_t**2)
23     else:
24         return 1.0
25
26 # 计算效用的函数
27 def calculate_utility(t, survival_func):
28     if t < 0:
29         return float("inf")
30     closest_time_idx = np.abs(survival_func.index - t).argmin()
31     actual_time = survival_func.index[closest_time_idx]
32     p_t = 1 - survival_func.loc[actual_time, "s_t"]
33     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
34
35 # 数据读取与预处理
36 df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
37 df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
38 df_always_can_test = pd.read_excel(
39     "./python_code/bmi_Y_always_can_test_result.xlsx"
40 )
41
42 # 合并数据并添加类别标识
43 df_all = pd.concat(
44     [
45         df_middle.assign(category="middle"),
46         df_cannot_test.assign(category="cannot"),
47         df_always_can_test.assign(category="always_can"),
48     ],
49     ignore_index=True,
50 )
51
52 df_all["duration"] = 0.0
53 df_all["event_observed"] = 0
54
55 # 根据不同类别设置duration和event_observed
56 df_all.loc[df_all["category"] == "cannot", "duration"] = df_all["最晚不达标天数"]
57 df_all.loc[df_all["category"] == "cannot", "event_observed"] = 0
58
59 df_all.loc[df_all["category"] == "middle", "duration"] = df_all["预测达标天数"]
60 df_all.loc[df_all["category"] == "middle", "event_observed"] = 1
61
62 df_all.loc[df_all["category"] == "always_can", "duration"] = 0.1
63 df_all.loc[df_all["category"] == "always_can", "event_observed"] = 1
64
65 # BMI分类处理
66 def categorize_bmi(bmi):
67     if bmi < 30.26:
68         return "<30.26"
69     elif 30.26 <= bmi < 32.30:
70         return "30.26-32.30"
71     elif 32.30 <= bmi < 34.92:
72         return "32.30-34.92"
73     elif 34.92 <= bmi < 39.49:
74         return "34.92-39.49"
75     else:

```

```

76         return ">39.49"
77
78     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
79
80     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
81
82     results_table = []
83
84     # 按BMI分组进行分析
85     for bmi_cat in bmi_categories:
86
87         df_bmi = df_all[df_all["bmi_category"] == bmi_cat].copy()
88
89         if df_bmi.empty:
90             results_table.append(
91                 {
92                     "BMI分组": bmi_cat,
93                     "最优时点(天)": "无数据",
94                     "最小效用值": "无数据",
95                     "风险水平": "无数据",
96                     "样本量": 0,
97                 }
98             )
99             continue
100
101         print(f"--- 正在分析 BMI 区间: {bmi_cat} ---")
102
103         # 生存分析
104         # 使用Kaplan-Meier估计生存函数
105         kmf = KaplanMeierFitter()
106         kmf.fit(durations=df_bmi["duration"], event_observed=df_bmi["event_observed"])
107
108         s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
109
110         # 优化效用函数
111         # 定义目标函数
112         def objective(t):
113             return calculate_utility(t, s_t)
114
115         # 寻找最小化效用函数的时间点
116         result = minimize_scalar(
117             objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
118         )
119
120         # 提取优化结果
121         optimal_time = result.x
122         optimal_utility = result.fun
123         risk_level = 1 / optimal_utility if optimal_utility > 0 else float("inf")
124         sample_size = len(df_bmi)
125
126         print(f"\n最优预测时间点分析:")
127         print(f" - 最优时间点: {optimal_time:.1f} 天")
128         print(f" - 最小效用值: {optimal_utility:.4f}")
129         print(f" - 风险水平: {risk_level:.4f}")
130         print(f" - 样本量: {sample_size}")

```

```

131
132
133 # 可视化分析结果
134 plt.figure(figsize=(14, 8))
135 ax = plt.gca()
136
137 # 绘制累积达标函数
138 (1 - kmf.survival_function_).plot(
139     ax=ax,
140     label=f"累积达标函数 F(t) ({bmi_cat})",
141     color="seagreen",
142     linewidth=2.5,
143 )
144
145 # 计算并绘制效用函数
146 time_points = np.linspace(0, df_bmi["duration"].max(), 200)
147 utilities = [calculate_utility(t, s_t) for t in time_points]
148 ax_twin = ax.twinx()
149 ax_twin.plot(
150     time_points, utilities, "--", color="red", label="效用函数 E(t)", alpha=0.6
151 )
152
153 ax_twin.axvline(x=optimal_time, color="purple", linestyle="--", linewidth=1)
154 ax_twin.text(
155     optimal_time + 2,
156     min(utilities) + 0.1,
157     f"最优时间点: {optimal_time:.1f}天\n效用值: {optimal_utility:.3f}",
158     color="purple",
159     fontsize=10,
160 )
161
162 ax.axvspan(0, 84, facecolor="green", alpha=0.15, label="早期阶段 (<84天)")
163 ax.axvspan(84, 189, facecolor="orange", alpha=0.15, label="过渡阶段 (84-189天)")
164 ax.axvspan(
165     189,
166     df_all["duration"].max() + 10,
167     facecolor="red",
168     alpha=0.15,
169     label="晚期阶段 (>189天)",
170 )
171
172 ax.set_title(f"BMI区间 {bmi_cat} 达标情况与效用分析", fontsize=16)
173 ax.set_xlabel("天数", fontsize=12)
174 ax.set_ylabel("累积达标比例", fontsize=12)
175 ax_twin.set_ylabel("效用值", fontsize=12)
176
177 lines1, labels1 = ax.get_legend_handles_labels()
178 lines2, labels2 = ax_twin.get_legend_handles_labels()
179 ax.legend(lines1 + lines2, labels1 + labels2, loc="upper left")
180
181 ax.grid(True, alpha=0.3)
182 ax.set_ylim(0, 1.05)
183 ax_twin.set_ylim(0, max(utilities) * 1.2)
184 ax.set_xlim(-5, df_all["duration"].max() + 10)
185

```

```

186     filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(">", "gt").replace("-", "_")}
        _utility_analysis.png'
187     plt.savefig(filename, dpi=300, bbox_inches="tight")
188     plt.close()
189
190     results_table.append(
191         {
192             "BMI 分组": bmi_cat,
193             "最优时点(天)": f"{optimal_time:.1f}",
194             "最小效用值": f"{optimal_utility:.4f}",
195             "风险水平": f"{risk_level:.4f}",
196             "样本量": sample_size,
197         }
198     )
199
200     print(f"\n图表已保存至: {filename}")
201     print("\n" + "=" * 50 + "\n")
202
203     # 结果输出与保存
204     print("\n\n=== 各BMI分组NIPT时点计算结果 ===")
205     print("BMI 分组\t\t最优时点(天)\t\t最小效用值\t\t风险水平\t\t样本量")
206     print("-" * 70)
207
208     for result in results_table:
209         print(
210             f"{result['BMI 分组']}\t\t{result['最优时点(天)']}\t\t{result['最小效用值']}\t\t{result['风险水
                平']}\t\t{result['样本量']}"
211         )
212
213     # 保存结果到Excel
214     results_df = pd.DataFrame(results_table)
215     results_df.to_excel("./python_code/NIPT_optimal_times_results.xlsx", index=False)
216     print(f"\n结果表格已保存至: ./python_code/NIPT_optimal_times_results.xlsx")
217
218     return results_df
219
220 # 执行
221 results = analyze_with_dynamic_utility()

```

Listing 3: C(2) 测试误差.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from lifelines import KaplanMeierFitter
5  from scipy.optimize import minimize_scalar
6  from scipy.stats import norm
7  import re
8
9  # 设置中文字体支持
10 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
11 plt.rcParams["axes.unicode_minus"] = False
12
13
14 def analyze_with_error_simulation(num_simulations=100, error_std=0.01):

```

```

15 # 定义效用函数
16 def get_rearly(t):
17     return 2.0 * np.exp(-t / 50)
18
19 def get_rlate(t):
20     if t < 84:
21         return 0.1
22     elif t <= 189:
23         normalized_t = (t - 84) / (189 - 84)
24         return 0.1 + 0.9 * (normalized_t**2)
25     else:
26         return 1.0
27
28 def calculate_utility(t, survival_func):
29     if t < 0:
30         return float("inf")
31     closest_time_idx = np.abs(survival_func.index - t).argmin()
32     actual_time = survival_func.index[closest_time_idx]
33     p_t = 1 - survival_func.loc[actual_time, "s_t"]
34     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
35
36 # 辅助函数：清理文件名中的非法字符
37 def clean_filename(name):
38     return re.sub(r'[\<>:"/\|?*]', "_", name)
39
40 # 数据加载与预处理
41 df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
42 df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
43 df_always_can_test = pd.read_excel(
44     "./python_code/bmi_Y_always_can_test_result.xlsx"
45 )
46
47 # 合并数据并添加类别标识
48 df_all = pd.concat(
49     [
50         df_middle.assign(category="middle"),
51         df_cannot_test.assign(category="cannot"),
52         df_always_can_test.assign(category="always_can"),
53     ],
54     ignore_index=True,
55 )
56
57 # 初始化duration和event_observed列
58 df_all["duration"] = 0.0
59 df_all["event_observed"] = 0
60
61 # 根据不同类别设置duration和event_observed
62 df_all.loc[df_all["category"] == "cannot", "duration"] = df_all["最晚不达标天数"]
63 df_all.loc[df_all["category"] == "cannot", "event_observed"] = 0
64
65 df_all.loc[df_all["category"] == "middle", "duration"] = df_all["预测达标天数"]
66 df_all.loc[df_all["category"] == "middle", "event_observed"] = 1
67
68 df_all.loc[df_all["category"] == "always_can", "duration"] = 0.1
69 df_all.loc[df_all["category"] == "always_can", "event_observed"] = 1

```



```

70
71 # BMI分类处理
72 def categorize_bmi(bmi):
73     if bmi < 30.26:
74         return "<30.26"
75     elif 30.26 <= bmi < 32.30:
76         return "30.26-32.30"
77     elif 32.30 <= bmi < 34.92:
78         return "32.30-34.92"
79     elif 34.92 <= bmi < 39.49:
80         return "34.92-39.49"
81     else:
82         return ">39.49"
83
84 df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
85 bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
86
87 # 初始化结果存储字典
88 results = {
89     bmi_cat: {"times": [], "utilities": [], "risks": []}
90     for bmi_cat in bmi_categories
91 }
92
93 # 原始数据分析
94 original_results = {}
95 for bmi_cat in bmi_categories:
96     df_bmi = df_all[df_all["bmi_category"] == bmi_cat].copy()
97     if df_bmi.empty:
98         continue
99
100 # 使用Kaplan-Meier估计生存函数
101 kmf = KaplanMeierFitter()
102 kmf.fit(durations=df_bmi["duration"], event_observed=df_bmi["event_observed"])
103 s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
104
105 # 优化效用函数
106 def objective(t):
107     return calculate_utility(t, s_t)
108
109 result = minimize_scalar(
110     objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
111 )
112
113 # 提取原始结果
114 original_time = result.x
115 original_utility = result.fun
116 original_risk = 1 / original_utility if original_utility > 0 else float("inf")
117
118 original_results[bmi_cat] = {
119     "time": original_time,
120     "utility": original_utility,
121     "risk": original_risk,
122 }
123
124 # 打印原始结果

```

```

125     print(f"BMI 区间 {bmi_cat} 原始结果:")
126     print(f"    最优时点: {original_time:.2f} 天")
127     print(f"    最小效用值: {original_utility:.4f}")
128     print(f"    风险水平: {original_risk:.4f}")
129
130 # 误差模拟分析
131 for simulation in range(num_simulations):
132     print(f"正在进行第 {simulation+1} 次模拟...")
133
134     # 创建模拟数据副本
135     df_simulated = df_all.copy()
136     np.random.seed(simulation)
137
138     # 为不同类别数据添加误差
139     mask_middle = df_simulated["category"] == "middle"
140     mask_always = df_simulated["category"] == "always_can"
141
142     if sum(mask_middle) > 0:
143         error_middle = np.random.normal(
144             0,
145             error_std * df_simulated.loc[mask_middle, "预测达标天数"].mean(),
146             size=sum(mask_middle),
147         )
148         df_simulated.loc[mask_middle, "duration"] += error_middle
149
150     if sum(mask_always) > 0:
151         error_always = np.random.normal(
152             0,
153             error_std * df_simulated.loc[mask_always, "最早达标天数"].mean(),
154             size=sum(mask_always),
155         )
156         df_simulated.loc[mask_always, "duration"] += error_always
157
158     mask_cannot = df_simulated["category"] == "cannot"
159     if sum(mask_cannot) > 0:
160         error_cannot = np.random.normal(
161             0,
162             error_std * df_simulated.loc[mask_cannot, "最晚不达标天数"].mean(),
163             size=sum(mask_cannot),
164         )
165         df_simulated.loc[mask_cannot, "duration"] += error_cannot
166
167     # 确保duration为正数
168     df_simulated["duration"] = df_simulated["duration"].clip(lower=0.1)
169
170     # 对每个BMI分类进行分析
171     for bmi_cat in bmi_categories:
172         df_bmi = df_simulated[df_simulated["bmi_category"] == bmi_cat].copy()
173         if df_bmi.empty:
174             continue
175
176         # 使用Kaplan-Meier估计生存函数
177         kmf = KaplanMeierFitter()
178         kmf.fit(
179             durations=df_bmi["duration"], event_observed=df_bmi["event_observed"]

```

```

180     )
181     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
182
183     # 优化效用函数
184     def objective(t):
185         return calculate_utility(t, s_t)
186
187     result = minimize_scalar(
188         objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
189     )
190
191     # 存储模拟结果
192     optimal_time = result.x
193     optimal_utility = result.fun
194     risk = 1 / optimal_utility if optimal_utility > 0 else float("inf")
195
196     results[bmi_cat]["times"].append(optimal_time)
197     results[bmi_cat]["utilities"].append(optimal_utility)
198     results[bmi_cat]["risks"].append(risk)
199
200     # 结果分析与可视化
201     print("\n=== 检测误差影响分析 ===")
202     for bmi_cat in bmi_categories:
203         if results[bmi_cat]["times"]:
204             # 计算统计量
205             times = np.array(results[bmi_cat]["times"])
206             utilities = np.array(results[bmi_cat]["utilities"])
207             risks = np.array(results[bmi_cat]["risks"])
208
209             mean_time = times.mean()
210             std_time = times.std()
211             time_confidence_interval = norm.interval(
212                 0.95, loc=mean_time, scale=std_time / np.sqrt(len(times))
213             )
214
215             mean_utility = utilities.mean()
216             std_utility = utilities.std()
217
218             mean_risk = risks.mean()
219             std_risk = risks.std()
220
221             # 获取原始结果
222             original_time = original_results.get(bmi_cat, {}).get("time", 0)
223             original_utility = original_results.get(bmi_cat, {}).get("utility", 0)
224             original_risk = original_results.get(bmi_cat, {}).get("risk", 0)
225
226             # 打印分析结果
227             print(f"BMI 区间 {bmi_cat}:")
228             print(f"    原始最优时点: {original_time:.2f} 天")
229             print(f"    模拟最优时点均值: {mean_time:.2f} 天")
230             print(f"    标准差: {std_time:.2f} 天")
231             print(
232                 f"    95%置信区间: ({time_confidence_interval[0]:.2f}, {time_confidence_interval[1]:.2f})"
233                 f"    天"
234             )

```

```

234     print(f" 原始最小效用值: {original_utility:.4f}")
235     print(f" 模拟最小效用值均值: {mean_utility:.4f}")
236     print(f" 效用值标准差: {std_utility:.4f}")
237     print(f" 原始风险水平: {original_risk:.4f}")
238     print(f" 模拟风险水平均值: {mean_risk:.4f}")
239     print(f" 风险水平标准差: {std_risk:.4f}")
240
241     # 创建可视化图表
242     plt.figure(figsize=(12, 8))
243
244     # 最优时点分布
245     plt.subplot(2, 2, 1)
246     plt.hist(times, bins=20, alpha=0.7, edgecolor="black")
247     plt.axvline(
248         mean_time,
249         color="r",
250         linestyle="--",
251         label=f"模拟平均时点: {mean_time:.2f}",
252     )
253     plt.axvline(
254         original_time,
255         color="b",
256         linestyle="-",
257         label=f"原始最优时点: {original_time:.2f}",
258     )
259     plt.axvline(
260         time_confidence_interval[0],
261         color="g",
262         linestyle=":",
263         label="95%置信区间",
264     )
265     plt.axvline(time_confidence_interval[1], color="g", linestyle=":")
266     plt.xlabel("最优NIPT时点 (天)")
267     plt.ylabel("频数")
268     plt.title(f"BMI 区间 {bmi_cat} - 最优时点分布")
269     plt.legend()
270     plt.grid(True, alpha=0.3)
271
272     # 最小效用值分布
273     plt.subplot(2, 2, 2)
274     plt.hist(utilities, bins=20, alpha=0.7, edgecolor="black", color="orange")
275     plt.axvline(
276         mean_utility,
277         color="r",
278         linestyle="--",
279         label=f"模拟平均效用值: {mean_utility:.4f}",
280     )
281     plt.axvline(
282         original_utility,
283         color="b",
284         linestyle="-",
285         label=f"原始最小效用值: {original_utility:.4f}",
286     )
287     plt.xlabel("最小效用值")
288     plt.ylabel("频数")

```

```

289 plt.title(f"BMI 区间 {bmi_cat} - 最小效用值分布")
290 plt.legend()
291 plt.grid(True, alpha=0.3)
292
293 # 时点与效用值关系
294 plt.subplot(2, 2, 3)
295 plt.scatter(times, utilities, alpha=0.6)
296 plt.axvline(original_time, color="b", linestyle="--", label="原始最优时点")
297 plt.axhline(
298     original_utility, color="b", linestyle="--", label="原始最小效用值"
299 )
300 plt.xlabel("最优时点 (天)")
301 plt.ylabel("最小效用值")
302 plt.title(f"BMI 区间 {bmi_cat} - 时点与效用值关系")
303 plt.legend()
304 plt.grid(True, alpha=0.3)
305
306 # 风险水平分布
307 plt.subplot(2, 2, 4)
308 plt.hist(risks, bins=20, alpha=0.7, edgecolor="black", color="red")
309 plt.axvline(
310     mean_risk,
311     color="r",
312     linestyle="--",
313     label=f"模拟平均风险: {mean_risk:.4f}",
314 )
315 plt.axvline(
316     original_risk,
317     color="b",
318     linestyle="--",
319     label=f"原始风险: {original_risk:.4f}",
320 )
321 plt.xlabel("风险水平")
322 plt.ylabel("频数")
323 plt.title(f"BMI 区间 {bmi_cat} - 风险水平分布")
324 plt.legend()
325 plt.grid(True, alpha=0.3)
326
327 # 保存图表
328 plt.tight_layout()
329 safe_bmi_cat = clean_filename(bmi_cat)
330 filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.png"
331 plt.savefig(filename, dpi=300, bbox_inches="tight")
332 plt.close()
333
334 # 保存数据到CSV
335 data_df = pd.DataFrame({"time": times, "utility": utilities, "risk": risks})
336 csv_filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.csv"
337 data_df.to_csv(csv_filename, index=False)
338
339 print(f" 图表已保存至: {filename}")
340 print(f" 数据已保存至: {csv_filename}")
341 else:
342     print(f"BMI 区间 {bmi_cat}: 无数据")
343

```

```

344
345 # 执行误差模拟分析
346 analyze_with_error_simulation(num_simulations=100, error_std=0.05)

```

Listing 4: C(2) 分类画图.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import matplotlib.font_manager as fm
5  from matplotlib import rcParams
6  from scipy import stats
7  import re
8
9  # 设置中文字体支持
10 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
11 plt.rcParams["axes.unicode_minus"] = False
12
13 # 生成每个BMI区间的堆叠分布图
14 def generate_stacked_bmi_mixed_charts():
15     # 加载数据文件
16     df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
17     df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
18     df_always_can_test = pd.read_excel(
19         "./python_code/bmi_Y_always_can_test_result.xlsx"
20     )
21
22     # 添加类别标识
23     df_middle["category"] = "middle"
24     df_cannot_test["category"] = "cannot"
25     df_always_can_test["category"] = "always_can"
26
27     # 合并所有数据
28     df_all = pd.concat(
29         [df_middle, df_cannot_test, df_always_can_test], ignore_index=True
30     )
31
32     # 定义BMI分类函数
33     def categorize_bmi(bmi):
34         if bmi < 30.26:
35             return "<30.26"
36         elif 30.26 <= bmi < 32.30:
37             return "30.26-32.30"
38         elif 32.30 <= bmi < 34.92:
39             return "32.30-34.92"
40         elif 34.92 <= bmi < 39.49:
41             return "34.92-39.49"
42         else:
43             return ">39.49"
44
45     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
46     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
47
48     # 定义类别、颜色和标签
49     categories = ["cannot", "middle", "always_can"]

```

```

50 colors = {"cannot": "lightcoral", "middle": "gold", "always_can": "seagreen"}
51 labels = {"cannot": "不能达标", "middle": "中间达标", "always_can": "始终达标"}
52
53 # 为每个BMI区间生成图表
54 for bmi_cat in bmi_categories:
55     df_bmi = df_all[df_all["bmi_category"] == bmi_cat]
56
57     if df_bmi.empty:
58         print(f"警告: BMI区间 {bmi_cat} 没有数据")
59         continue
60
61     # 创建图表
62     fig, ax1 = plt.subplots(figsize=(12, 8))
63
64     # 收集所有天数数据
65     all_days = []
66     days_data = {}
67
68     # 处理每个类别的数据
69     for category in categories:
70         df_category = df_bmi[df_bmi["category"] == category]
71
72         if not df_category.empty:
73             if category == "cannot":
74                 days = df_category["最晚不达标天数"].tolist()
75             elif category == "middle":
76                 days = df_category["预测达标天数"].tolist()
77             else:
78                 days = df_category["最早达标天数"].tolist()
79
80             days_data[category] = days
81             all_days.extend(days)
82
83     # 准备堆叠数据
84     stacked_data = []
85     stacked_labels = []
86     stacked_colors = []
87
88     if "cannot" in days_data:
89         stacked_data.append(days_data["cannot"])
90         stacked_labels.append(labels["cannot"])
91         stacked_colors.append(colors["cannot"])
92
93     if "middle" in days_data:
94         stacked_data.append(days_data["middle"])
95         stacked_labels.append(labels["middle"])
96         stacked_colors.append(colors["middle"])
97
98     if "always_can" in days_data:
99         stacked_data.append(days_data["always_can"])
100         stacked_labels.append(labels["always_can"])
101         stacked_colors.append(colors["always_can"])
102
103     # 绘制堆叠直方图
104     if stacked_data:

```

```

105         if len(all_days) > 0:
106             data_range = max(all_days) - min(all_days)
107             bins_count = min(100, max(50, int(data_range / 3)))
108         else:
109             bins_count = 50
110
111         n, bins, patches = ax1.hist(
112             stacked_data,
113             bins=bins_count,
114             stacked=True,
115             color=stacked_colors,
116             label=stacked_labels,
117             alpha=0.95,
118             edgecolor="black",
119             linewidth=0.1,
120         )
121
122         # 设置图表属性
123         ax1.set_xlabel("天数", fontsize=12)
124         ax1.set_ylabel("人数", fontsize=12)
125         ax1.set_title(
126             f"BMI区间 {bmi_cat} 的孕妇Y染色体达标情况分布（堆叠图）", fontsize=14
127         )
128         ax1.legend(fontsize=10)
129         ax1.grid(True, alpha=0.3)
130
131         # 添加概率密度曲线
132         if len(all_days) > 1:
133             ax2 = ax1.twinx()
134
135             try:
136                 kde = stats.gaussian_kde(all_days)
137                 x_range = np.linspace(min(all_days), max(all_days), 1000)
138                 kde_values = kde(x_range)
139
140                 ax2.plot(
141                     x_range,
142                     kde_values,
143                     color="dodgerblue",
144                     linewidth=2,
145                     linestyle="--",
146                     label="总体概率密度",
147                 )
148
149                 ax2.set_ylabel("概率密度", fontsize=12)
150                 ax2.legend(bbox_to_anchor=(1.0, 0.9), fontsize=10)
151             except Exception as e:
152                 print(f"BMI区间 {bmi_cat} 计算概率密度时出错: {e}")
153
154         # 保存图表
155         plt.tight_layout()
156         filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(">", "gt").replace("-", "_")}_stacked_distribution.png'
157         plt.savefig(filename, dpi=300, bbox_inches="tight")
158         plt.close()

```



```

159
160     # 打印统计信息
161     print(f"BMI 区间 {bmi_cat} 统计:")
162     total_count = len(df_bmi)
163     for category in categories:
164         count = len(df_bmi[df_bmi["category"] == category])
165         percentage = (count / total_count) * 100 if total_count > 0 else 0
166         print(f"    {labels[category]}: {count} 人 ({percentage:.1f}%)")
167     print(f"    总计: {total_count} 人")
168     print()
169
170 # 生成总体堆叠柱状图
171 def generate_overall_stacked_bmi_chart():
172     # 加载数据文件
173     df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
174     df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
175     df_always_can_test = pd.read_excel(
176         "./python_code/bmi_Y_always_can_test_result.xlsx"
177     )
178
179     # 添加类别标识
180     df_middle["category"] = "middle"
181     df_cannot_test["category"] = "cannot"
182     df_always_can_test["category"] = "always_can"
183
184     # 合并所有数据
185     df_all = pd.concat(
186         [df_middle, df_cannot_test, df_always_can_test], ignore_index=True
187     )
188
189     # 定义BMI分类函数
190     def categorize_bmi(bmi):
191         if bmi < 30.26:
192             return "<30.26"
193         elif 30.26 <= bmi < 32.30:
194             return "30.26-32.30"
195         elif 32.30 <= bmi < 34.92:
196             return "32.30-34.92"
197         elif 34.92 <= bmi < 39.49:
198             return "34.92-39.49"
199         else:
200             return ">39.49"
201
202     # 应用BMI分类
203     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
204     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
205
206     # 定义类别、颜色和标签
207     categories = ["cannot", "middle", "always_can"]
208     colors = {"cannot": "red", "middle": "yellow", "always_can": "green"}
209     labels = {"cannot": "不能达标", "middle": "中间达标", "always_can": "始终达标"}
210
211     # 创建图表
212     fig, ax1 = plt.subplots(figsize=(15, 8))
213

```

```

214 # 按类别收集数据
215 all_data_by_category = {cat: [] for cat in categories}
216
217 # 处理每个BMI区间的数据
218 for bmi_cat in bmi_categories:
219     df_bmi = df_all[df_all["bmi_category"] == bmi_cat]
220
221     for category in categories:
222         df_category = df_bmi[df_bmi["category"] == category]
223
224         if not df_category.empty:
225             if category == "cannot":
226                 days = df_category["最晚不达标天数"].tolist()
227             elif category == "middle":
228                 days = df_category["预测达标天数"].tolist()
229             else:
230                 days = df_category["最早达标天数"].tolist()
231
232             labeled_days = [(day, bmi_cat) for day in days]
233             all_data_by_category[category].extend(labeled_days)
234
235 # 准备柱状图数据
236 x_positions = np.arange(len(bmi_categories))
237 bar_width = 0.6
238 bottom_values = np.zeros(len(bmi_categories))
239
240 # 绘制堆叠柱状图
241 for category in categories:
242     heights = []
243     for bmi_cat in bmi_categories:
244         count = len([x for x in all_data_by_category[category] if x[1] == bmi_cat])
245         heights.append(count)
246
247     ax1.bar(
248         x_positions,
249         heights,
250         bar_width,
251         bottom=bottom_values,
252         label=labels[category],
253         color=colors[category],
254         alpha=0.7,
255         edgecolor="black",
256         linewidth=0.5,
257     )
258
259     bottom_values = np.add(bottom_values, heights)
260
261 # 设置图表属性
262 ax1.set_xlabel("BMI 区间", fontsize=12)
263 ax1.set_ylabel("人数", fontsize=12)
264 ax1.set_title("各BMI区间孕妇Y染色体达标情况分布（堆叠柱状图）", fontsize=14)
265 ax1.set_xticks(x_positions)
266 ax1.set_xticklabels(bmi_categories)
267 ax1.legend(fontsize=10)
268 ax1.grid(True, alpha=0.3, axis="y")

```

```

269
270 # 保存图表
271 plt.tight_layout()
272 plt.savefig(
273     "./python_code/BMI_all_intervals_stacked_distribution_new.png",
274     dpi=300,
275     bbox_inches="tight",
276 )
277 plt.close()
278
279 print("总体堆叠柱状图已生成")
280
281
282 if __name__ == "__main__":
283     generate_overall_stacked_bmi_chart()
284     generate_stacked_bmi_mixed_charts()
285
286     print("\n图表生成完成!")

```

Listing 5: C(2) 聚类.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.cluster import KMeans
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.metrics import (
7      silhouette_score,
8      davies_bouldin_score,
9      calinski_harabasz_score,
10 )
11
12 # 设置中文字体支持
13 plt.rcParams["font.sans-serif"] = ["SimHei"]
14 plt.rcParams["axes.unicode_minus"] = False
15
16
17 # 执行K-means聚类分析
18 def perform_clustering(df, n_clusters=5):
19     X = df[["BMI"]].values
20     scaler = StandardScaler()
21     X_scaled = scaler.fit_transform(X)
22
23     kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
24     clusters = kmeans.fit_predict(X_scaled)
25
26     df["Cluster"] = clusters
27
28     centers_scaled = kmeans.cluster_centers_
29     centers_original = scaler.inverse_transform(centers_scaled)
30
31     return df, kmeans, scaler, centers_original, X_scaled
32
33
34 # 评估聚类效果

```

```

35 def evaluate_clustering(X, clusters):
36     silhouette_avg = silhouette_score(X, clusters)
37     dbi = davies_bouldin_score(X, clusters)
38     ch_index = calinski_harabasz_score(X, clusters)
39
40     print("\n聚类效果评估:")
41     print(f"轮廓系数 (Silhouette Score): {silhouette_avg:.4f}")
42     print(f"DBI指数 (Davies-Bouldin Index): {dbi:.4f}")
43     print(f"CH指数 (Calinski-Harabasz Index): {ch_index:.4f}")
44
45     return silhouette_avg, dbi, ch_index
46
47
48 # 打印各聚类的BMI范围统计信息
49 def print_cluster_ranges(df):
50     print("\n各聚类的BMI范围:")
51     print("-" * 50)
52     for i in range(5):
53         cluster_data = df[df["Cluster"] == i]["BMI"]
54         min_bmi = cluster_data.min()
55         max_bmi = cluster_data.max()
56         mean_bmi = cluster_data.mean()
57         count = len(cluster_data)
58
59         if mean_bmi < 18.5:
60             category = "偏瘦"
61         elif mean_bmi < 24:
62             category = "正常"
63         elif mean_bmi < 28:
64             category = "偏胖"
65         else:
66             category = "肥胖"
67
68         print(
69             f"聚类 {i}: {count:3d} 个样本, BMI范围: {min_bmi:.2f}~{max_bmi:.2f}, "
70             f"平均值: {mean_bmi:.2f} ({category})"
71         )
72
73     print("-" * 50)
74
75
76 # 可视化聚类结果
77 def visualize_results(df, centers_original, silhouette_avg, dbi, ch_index):
78     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
79
80     # 绘制各聚类BMI分布直方图
81     for i in range(5):
82         cluster_data = df[df["Cluster"] == i]["BMI"]
83         axes[0, 0].hist(cluster_data, alpha=0.6, bins=20, label=f"Cluster {i}")
84
85     axes[0, 0].axvline(x=18.5, color="r", linestyle="--", label="偏瘦阈值")
86     axes[0, 0].axvline(x=24, color="g", linestyle="--", label="正常阈值")
87     axes[0, 0].axvline(x=28, color="orange", linestyle="--", label="偏胖阈值")
88     axes[0, 0].set_xlabel("BMI")
89     axes[0, 0].set_ylabel("频数")

```

```

90 axes[0, 0].set_title("各聚类BMI分布")
91 axes[0, 0].legend()
92 axes[0, 0].grid(True, alpha=0.3)
93
94 # 绘制各聚类中心BMI值柱状图
95 cluster_labels = [f"Cluster {i}" for i in range(5)]
96 axes[0, 1].bar(
97     cluster_labels,
98     centers_original.flatten(),
99     color=["blue", "green", "red", "purple", "orange"],
100 )
101 axes[0, 1].set_ylabel("BMI值")
102 axes[0, 1].set_title("各聚类中心BMI值")
103 axes[0, 1].grid(True, alpha=0.3)
104
105 # 绘制BMI与达标天数的散点图
106 for i in range(5):
107     cluster_data = df[df["Cluster"] == i]
108     axes[1, 0].scatter(
109         cluster_data["BMI"],
110         cluster_data["对应达标天数"],
111         alpha=0.6,
112         label=f"Cluster {i}",
113     )
114
115 axes[1, 0].set_xlabel("BMI")
116 axes[1, 0].set_ylabel("达标天数")
117 axes[1, 0].set_title("BMI与达标天数的关系")
118 axes[1, 0].legend()
119 axes[1, 0].grid(True, alpha=0.3)
120
121
122 # 保存分析结果到Excel文件
123 def save_results(df, centers_original, silhouette_avg, dbi, ch_index):
124     summary_data = {
125         "聚类编号": range(5),
126         "聚类中心(BMI)": centers_original.flatten(),
127         "样本数量": [len(df[df["Cluster"] == i]) for i in range(5)],
128         "BMI最小值": [df[df["Cluster"] == i]["BMI"].min() for i in range(5)],
129         "BMI最大值": [df[df["Cluster"] == i]["BMI"].max() for i in range(5)],
130         "BMI平均值": [df[df["Cluster"] == i]["BMI"].mean() for i in range(5)],
131     }
132     summary_df = pd.DataFrame(summary_data)
133
134     eval_df = pd.DataFrame(
135         {
136             "评估指标": ["轮廓系数", "DBI指数", "CH指数"],
137             "值": [silhouette_avg, dbi, ch_index],
138         }
139     )
140
141     with pd.ExcelWriter("聚类分析结果.xlsx") as writer:
142         df.to_excel(writer, sheet_name="原始数据与聚类结果", index=False)
143         summary_df.to_excel(writer, sheet_name="聚类摘要", index=False)
144         eval_df.to_excel(writer, sheet_name="评估指标", index=False)

```

```

145
146     print("\n结果已保存到 '聚类分析结果.xlsx'")
147
148
149 # 主函数，整合所有分析步骤
150 def main():
151     # 加载数据
152     file_path = "python_code/bmi_Y_result.xlsx"
153     df = pd.read_excel(file_path)
154
155     # 执行聚类分析
156     df_result, kmeans, scaler, centers_original, X_scaled = perform_clustering(df)
157
158     # 评估聚类效果
159     silhouette_avg, dbi, ch_index = evaluate_clustering(
160         X_scaled, df_result["Cluster"].values
161     )
162
163     # 打印聚类统计信息
164     print_cluster_ranges(df_result)
165
166     # 可视化结果
167     visualize_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
168
169     # 保存结果
170     save_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
171
172
173 # 程序入口
174 if __name__ == "__main__":
175     main()

```

Listing 6: C(3)preprocessing.py

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.cluster import KMeans
4  from sklearn.preprocessing import StandardScaler
5  from lifelines import KaplanMeierFitter
6  from scipy.optimize import minimize_scalar
7  import matplotlib.pyplot as plt
8  import re
9
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # 加载附件数据并进行预处理
14 def load_and_preprocess_data():
15
16
17     df = pd.read_excel('./python_code/附件.xlsx', sheet_name=0)
18
19     def weeks_to_days(weeks_str):
20         match = re.match(r'(\d+)[wW](?:\+(\d+))?', str(weeks_str), re.IGNORECASE)
21         if match:

```

```

22         weeks = int(match.group(1))
23         days = int(match.group(2)) if match.group(2) else 0
24         return weeks * 7 + days
25     else:
26         print(f"无法解析孕周格式: {weeks_str}")
27         return None
28
29 df['检测孕周_天数'] = df['检测孕周'].apply(weeks_to_days)
30 # 提取男胎数据 (Y染色体浓度非空)
31 male_data = df[df['Y染色体浓度'].notna()].copy()
32 print(f"男胎数据量: {len(male_data)}")
33
34 # 选择特征: BMI、身高、体重、年龄
35 features = male_data[['孕妇代码', '年龄', '身高', '体重', '孕妇BMI', 'Y染色体浓度', 'Y染色体的Z值',
36                        '检测孕周_天数']].copy()
37
38 # 处理缺失值: 均值插补
39 for col in features.columns:
40     if col != '孕妇代码': # 跳过非数值列
41         if features[col].isnull().sum() > 0:
42             mean_val = features[col].mean()
43             features[col].fillna(mean_val, inplace=True)
44             print(f"列 '{col}' 有 {features[col].isnull().sum()} 个缺失值, 已用均值 {mean_val:.2f} 填充")
45
46 # 检查并处理可能的非数值数据
47 for col in features.columns:
48     if col != '孕妇代码': # 跳过非数值列
49         # 尝试转换为数值类型
50         features[col] = pd.to_numeric(features[col], errors='coerce')
51         # 再次检查并填充可能出现的NaN
52         if features[col].isnull().sum() > 0:
53             mean_val = features[col].mean()
54             features[col].fillna(mean_val, inplace=True)
55             print(f"列 '{col}' 转换后有缺失值, 已用均值 {mean_val:.2f} 填充")
56
57 # 使用Z-score标准化特征 (排除孕妇代码、Y染色体浓度和Y染色体的Z值)
58 scaler = StandardScaler() # StandardScaler默认使用Z-score标准化
59 feature_cols = ['年龄', '身高', '体重', '孕妇BMI', '检测孕周_天数']
60 features_scaled = scaler.fit_transform(features[feature_cols])
61 features_scaled_df = pd.DataFrame(features_scaled,
62                                   columns=[f'{col}_标准化' for col in feature_cols])
63
64 # 只保留需要的列: 孕妇代码、原始特征、标准化特征、Y染色体浓度和Y染色体的Z值
65 final_data = pd.concat([
66     features[['孕妇代码', 'Y染色体浓度', 'Y染色体的Z值']],
67     features[feature_cols],
68     features_scaled_df
69 ], axis=1)
70
71 # 根据Y染色体浓度判断能否检测
72 final_data['能否检测'] = final_data['Y染色体浓度'].apply(lambda x: 1 if x > 0.04 else 0)
73
74 can_code = []
75 df_codes = df['孕妇代码'].unique()

```

```

75     for code in df_codes:
76         data = df[df['孕妇代码'] == code].sort_values(by='检测孕周_天数')
77         first_ok_idx = None
78
79         for idx, row in data.iterrows():
80             if row['Y染色体浓度'] >= 0.04:
81                 first_ok_idx = idx
82                 break
83
84         if first_ok_idx is not None:
85             later_data = data.loc[first_ok_idx:]
86             if (later_data['Y染色体浓度'] < 0.04).any():
87                 continue
88             else:
89                 can_code.append(code)
90
91     final_data = final_data[final_data['孕妇代码'].isin(can_code)]
92
93
94     # 保存处理后的数据
95     try:
96         final_data.to_excel('./python_code/Q3/Q3数据预处理.xlsx', index=False)
97         print("预处理后的数据已保存至: ./python_code/Q3/Q3数据预处理.xlsx")
98     except Exception as e:
99         print(f"保存文件时出错: {e}")
100    return None
101
102    # 打印数据基本信息
103    print("\n=== 数据预处理完成 ===")
104    print(f"处理后的数据形状: {final_data.shape}")
105    print("\n特征统计信息:")
106    print(final_data.describe())
107
108    return final_data
109
110    # 运行数据预处理
111    processed_data = load_and_preprocess_data()

```

Listing 7: C(3) 聚类.py

```

1
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.cluster import KMeans
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
8
9  # 设置中文字体支持
10 plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
11 plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
12
13 # 执行聚类分析
14 def perform_clustering(df, n_clusters=5):
15     """

```



```

16     执行K-means聚类分析
17     """
18     # 提取BMI数据并标准化
19     X = df[['孕妇BMI']].values
20     scaler = StandardScaler()
21     X_scaled = scaler.fit_transform(X)
22
23     # 执行K-means聚类
24     kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
25     clusters = kmeans.fit_predict(X_scaled)
26
27     # 将聚类结果添加到数据框中
28     df['Cluster'] = clusters
29
30     # 计算聚类中心（原始尺度）
31     centers_scaled = kmeans.cluster_centers_
32     centers_original = scaler.inverse_transform(centers_scaled)
33
34     return df, kmeans, scaler, centers_original, X_scaled
35
36 # 评估聚类效果
37 def evaluate_clustering(X, clusters):
38     """
39     计算聚类评估指标
40     """
41     silhouette_avg = silhouette_score(X, clusters)
42     dbi = davies_bouldin_score(X, clusters)
43     ch_index = calinski_harabasz_score(X, clusters)
44
45     print("\n聚类效果评估:")
46     print(f"轮廓系数 (Silhouette Score): {silhouette_avg:.4f}")
47     print(f"DBI指数 (Davies-Bouldin Index): {dbi:.4f}")
48     print(f"CH指数 (Calinski-Harabasz Index): {ch_index:.4f}")
49
50     return silhouette_avg, dbi, ch_index
51
52 # 打印分类的BMI范围
53 def print_cluster_ranges(df):
54     """
55     打印每个聚类的BMI范围
56     """
57     print("\n各聚类的BMI范围:")
58     print("-" * 50)
59     for i in range(5):
60         cluster_data = df[df['Cluster'] == i]['孕妇BMI']
61         min_bmi = cluster_data.min()
62         max_bmi = cluster_data.max()
63         mean_bmi = cluster_data.mean()
64         count = len(cluster_data)
65
66         # 确定BMI分类标签
67         if mean_bmi < 18.5:
68             category = "偏瘦"
69         elif mean_bmi < 24:
70             category = "正常"

```

```

71         elif mean_bmi < 28:
72             category = "偏胖"
73         else:
74             category = "肥胖"
75
76         print(f"聚类 {i}: {count:3d} 个样本, BMI范围: {min_bmi:.2f}-{max_bmi:.2f}, "
77               f"平均值: {mean_bmi:.2f} ({category})")
78
79     print("-" * 50)
80
81 # 可视化结果
82 def visualize_results(df, centers_original, silhouette_avg, dbi, ch_index):
83     """
84     可视化聚类结果
85     """
86     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
87
88     # 聚类分布直方图
89     for i in range(5):
90         cluster_data = df[df['Cluster'] == i]['孕妇BMI']
91         axes[0, 0].hist(cluster_data, alpha=0.6, bins=20, label=f'Cluster {i}')
92
93         axes[0, 0].axvline(x=18.5, color='r', linestyle='--', label='偏瘦阈值')
94         axes[0, 0].axvline(x=24, color='g', linestyle='--', label='正常阈值')
95         axes[0, 0].axvline(x=28, color='orange', linestyle='--', label='偏胖阈值')
96         axes[0, 0].set_xlabel('BMI')
97         axes[0, 0].set_ylabel('频数')
98         axes[0, 0].set_title('各聚类BMI分布')
99         axes[0, 0].legend()
100        axes[0, 0].grid(True, alpha=0.3)
101
102    # 聚类中心比较
103    cluster_labels = [f'Cluster {i}' for i in range(5)]
104    axes[0, 1].bar(cluster_labels, centers_original.flatten(), color=['blue', 'green', 'red', 'purple',
105        'orange'])
106    axes[0, 1].set_ylabel('BMI值')
107    axes[0, 1].set_title('各聚类中心BMI值')
108    axes[0, 1].grid(True, alpha=0.3)
109
110    # 达标天数与BMI的关系
111    for i in range(5):
112        cluster_data = df[df['Cluster'] == i]
113        axes[1, 0].scatter(cluster_data['孕妇BMI'], cluster_data['检测孕周_天数'],
114            alpha=0.6, label=f'Cluster {i}')
115
116        axes[1, 0].set_xlabel('BMI')
117        axes[1, 0].set_ylabel('检测孕周_天数')
118        axes[1, 0].set_title('BMI与检测孕周_天数的关系')
119        axes[1, 0].legend()
120        axes[1, 0].grid(True, alpha=0.3)
121
122    plt.savefig('聚类分析可视化结果.png', dpi=300, bbox_inches='tight')
123    print("可视化图表已保存为 '聚类分析可视化结果.png'")
124

```

```

125
126
127 # 保存结果
128 def save_results(df, centers_original, silhouette_avg, dbi, ch_index):
129     """
130     保存聚类结果到Excel文件
131     """
132     # 创建结果摘要
133     summary_data = {
134         '聚类编号': range(5),
135         '聚类中心(BMI)': centers_original.flatten(),
136         '样本数量': [len(df[df['Cluster'] == i]) for i in range(5)],
137         'BMI最小值': [df[df['Cluster'] == i]['孕妇BMI'].min() for i in range(5)],
138         'BMI最大值': [df[df['Cluster'] == i]['孕妇BMI'].max() for i in range(5)],
139         'BMI平均值': [df[df['Cluster'] == i]['孕妇BMI'].mean() for i in range(5)]
140     }
141     summary_df = pd.DataFrame(summary_data)
142
143     # 添加评估指标
144     eval_df = pd.DataFrame({
145         '评估指标': ['轮廓系数', 'DBI指数', 'CH指数'],
146         '值': [silhouette_avg, dbi, ch_index]
147     })
148
149     # 保存到Excel
150     with pd.ExcelWriter('聚类分析结果.xlsx') as writer:
151         df.to_excel(writer, sheet_name='原始数据与聚类结果', index=False)
152         summary_df.to_excel(writer, sheet_name='聚类摘要', index=False)
153         eval_df.to_excel(writer, sheet_name='评估指标', index=False)
154
155     print("\n结果已保存到 '聚类分析结果.xlsx'")
156
157 # 主函数
158 def main():
159     # 请替换为您的Excel文件路径
160     file_path = "python_code\Q3\Q3数据预处理.xlsx"
161     df = pd.read_excel(file_path)
162     # 执行聚类分析
163     df_result, kmeans, scaler, centers_original, X_scaled = perform_clustering(df)
164
165     # 评估聚类效果
166     silhouette_avg, dbi, ch_index = evaluate_clustering(X_scaled, df_result['Cluster'].values)
167
168     # 打印分类的BMI范围
169     print_cluster_ranges(df_result)
170
171     # 可视化结果
172     visualize_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
173
174     # 保存结果
175     save_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
176
177 if __name__ == "__main__":
178     main()

```

Listing 8: C(3)final.py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestRegressor
7 from sklearn.linear_model import LinearRegression
8 from sklearn.model_selection import cross_val_score, KFold
9 from scipy.optimize import minimize_scalar
10 from lifelines import KaplanMeierFitter
11 import shap
12 from statsmodels.nonparametric.smoothers_lowess import lowess
13 import warnings
14 from scipy.stats import norm
15 import re
16 warnings.filterwarnings('ignore')
17
18 # 设置中文字体支持
19 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
20 plt.rcParams["axes.unicode_minus"] = False
21
22 # 建模分析不同BMI分组中年龄、身高、体重对NIPT最佳时点的影响
23 def analyze_nipt_optimal_time_with_advanced_modeling():
24     # 读取预处理数据
25     file_path = "python_code/Q3/Q3数据预处理.xlsx"
26     df = pd.read_excel(file_path)
27
28     # 定义BMI分组函数
29     def categorize_bmi(bmi):
30         if bmi < 30.01:
31             return "聚类0 (<30.01)"
32         elif 30.01 <= bmi < 32.18:
33             return "聚类1 (30.01-32.18)"
34         elif 32.18 <= bmi < 34.63:
35             return "聚类2 (32.18-34.63)"
36         elif 34.63 <= bmi < 37.93:
37             return "聚类3 (34.63-37.93)"
38         else:
39             return "聚类4 (37.93)"
40
41     df['bmi_category'] = df['孕妇BMI'].apply(categorize_bmi)
42
43     # 准备生存分析数据
44     survival_data = []
45
46     for code, group in df.groupby('孕妇代码'):
47         group = group.sort_values('检测孕周_天数')
48         # 找到首次达标的时间
49         first_reach_idx = group[group['Y染色体浓度'] >= 0.04].index
50         last_below_idx = group[group['Y染色体浓度'] < 0.04].index
51
52         if len(first_reach_idx) > 0:
53             # 有达标记录
54             first_reach_time = group.loc[first_reach_idx[0], '检测孕周_天数']

```

```

55     first_reach_concentration = group.loc[first_reach_idx[0], 'Y染色体浓度']
56
57     # 找到最后一次未达标的时间
58     if len(last_below_idx) > 0 and last_below_idx[-1] < first_reach_idx[0]:
59         last_below_time = group.loc[last_below_idx[-1], '检测孕周_天数']
60         last_below_concentration = group.loc[last_below_idx[-1], 'Y染色体浓度']
61
62         # 使用线性插值法计算达标时间
63         # 公式:  $t = t_0 + (t_1 - t_0) * (0.04 - c_0) / (c_1 - c_0)$ 
64         event_time = last_below_time + (first_reach_time - last_below_time) * \
65             (0.04 - last_below_concentration) / (first_reach_concentration -
66             last_below_concentration)
67
68     else:
69         # 如果没有之前的未达标记录, 使用首次达标时间
70         event_time = first_reach_time
71
72     event_observed = 1
73 else:
74     # 未达标, 使用最后一次检测时间作为删失时间
75     event_time = group.iloc[-1]['检测孕周_天数']
76     event_observed = 0
77
78 # 获取孕妇的基本信息 (取第一次检测的值)
79 base_info = group.iloc[0]
80
81 survival_data.append({
82     '孕妇代码': code,
83     '年龄': base_info['年龄'],
84     '身高': base_info['身高'],
85     '体重': base_info['体重'],
86     '孕妇BMI': base_info['孕妇BMI'],
87     'bmi_category': base_info['bmi_category'],
88     '达标时间': event_time,
89     '达标状态': event_observed
90 })
91
92 survival_df = pd.DataFrame(survival_data)
93
94 # 定义效用函数
95 def get_rearly(t):
96     """早期检测效用函数: 随时间指数衰减"""
97     return 2.0 * np.exp(-t / 50)
98
99 def get_rlate(t):
100     """晚期检测风险函数: 分段函数"""
101     if t < 84: # 12周
102         return 0.1
103     elif t <= 189: # 27周
104         normalized_t = (t - 84) / (189 - 84)
105         return 0.1 + 0.9 * (normalized_t**2)
106     else: # 超过27周
107         return 1.0
108
109 def calculate_utility(t, survival_func):
110     """计算总效用函数"""

```

```

109     if t < 0:
110         return float("inf")
111
112     # 找到最近的时间点
113     closest_time_idx = np.abs(survival_func.index - t).argmin()
114     actual_time = survival_func.index[closest_time_idx]
115
116     # 计算达标概率
117     p_t = 1 - survival_func.loc[actual_time, "s_t"]
118
119     # 计算总效用
120     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
121
122 # 分析每个BMI分组
123 bmi_categories = [
124     "聚类0 (<30.01)",
125     "聚类1 (30.01-32.18)",
126     "聚类2 (32.18-34.63)",
127     "聚类3 (34.63-37.93)",
128     "聚类4 (37.93)"
129 ]
130
131 results_table = []
132 model_performance = []
133 equations = [] # 存储每个分组的NIPT时点方程
134 optimal_times = {} # 存储每个分组的最佳时点
135
136 for bmi_cat in bmi_categories:
137     df_bmi = survival_df[survival_df['bmi_category'] == bmi_cat].copy()
138
139     if df_bmi.empty:
140         results_table.append({
141             "BMI分组": bmi_cat,
142             "最优时点(天)": "无数据",
143             "最小效用值": "无数据",
144             "风险水平": "无数据",
145             "样本量": 0,
146         })
147         continue
148
149     print(f"\n=== 正在分析 BMI 分组: {bmi_cat} ===")
150     print(f"样本量: {len(df_bmi)}")
151
152     # 计算KM生存曲线
153     kmf = KaplanMeierFitter()
154     kmf.fit(durations=df_bmi["达标时间"], event_observed=df_bmi["达标状态"])
155
156     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
157
158     # 优化找到最佳时点
159     def objective(t):
160         return calculate_utility(t, s_t)
161
162     result = minimize_scalar(
163         objective, bounds=(0, df_bmi["达标时间"].max()), method="bounded"

```

```

164 )
165
166 optimal_time = result.x
167 optimal_utility = result.fun
168 risk_level = 1 / optimal_utility if optimal_utility > 0 else float("inf")
169 sample_size = len(df_bmi)
170
171 # 存储最优时点
172 optimal_times[bmi_cat] = optimal_time
173
174 print(f"\n最优预测时间点分析:")
175 print(f" - 最优时间点: {optimal_time:.1f} 天 (约{optimal_time/7:.1f}周)")
176 print(f" - 最小效用值: {optimal_utility:.4f}")
177 print(f" - 风险水平: {risk_level:.4f}")
178
179 # 使用随机森林建模年龄、身高、体重对达标时间的影响
180 print("\n使用随机森林建模影响因素:")
181
182 # 重置索引以确保一致性
183 df_bmi_reset = df_bmi.reset_index(drop=True)
184
185 # 准备回归数据
186 X = df_bmi_reset[['年龄', '身高', '体重']]
187 y = df_bmi_reset['达标时间']
188
189 # 检查样本量是否足够进行建模
190 if len(X) < 10: # 至少需要10个样本
191     print("样本量不足, 使用线性回归建模")
192
193 # 使用线性回归
194 try:
195     # 使用线性回归
196     lr = LinearRegression()
197     lr.fit(X, y)
198
199     # 获取系数
200     coefficients = lr.coef_
201     intercept = lr.intercept_
202
203     # 生成回归方程
204     equation = f"T = {intercept:.2f} + {coefficients[0]:.2f}×年龄 + {coefficients[1]:.2f}×身
        高 + {coefficients[2]:.2f}×体重"
205
206     # 计算模型性能
207     y_pred = lr.predict(X)
208     model_r2 = lr.score(X, y)
209     model_mae = np.mean(np.abs(y - y_pred))
210
211     print(f"线性回归模型性能:")
212     print(f" - R²得分: {model_r2:.4f}")
213     print(f" - 平均绝对误差: {model_mae:.2f}天")
214     print(f"回归方程: {equation}")
215
216     # 计算特征重要性 (使用系数的绝对值)
217     feature_importance = np.abs(coefficients) / np.sum(np.abs(coefficients))

```

```

218     age_importance = feature_importance[0]
219     height_importance = feature_importance[1]
220     weight_importance = feature_importance[2]
221
222     print(f"特征重要性:")
223     print(f" - 年龄: {age_importance:.4f}")
224     print(f" - 身高: {height_importance:.4f}")
225     print(f" - 体重: {weight_importance:.4f}")
226
227     # 验证引入影响因素后的改进
228     y_mean = np.full_like(y_pred, y.mean())
229     mae_mean = np.mean(np.abs(y - y_mean))
230     improvement = (mae_mean - model_mae) / mae_mean * 100
231
232     print(f"\n模型验证结果:")
233     print(f" - 仅使用平均值预测的MAE: {mae_mean:.2f}天")
234     print(f" - 使用线性回归预测的MAE: {model_mae:.2f}天")
235     print(f" - 改进百分比: {improvement:.2f}%")
236
237     model_performance.append({
238         "BMI分组": bmi_cat,
239         "样本量": len(X),
240         "R²得分": model_r2,
241         "模型MAE": model_mae,
242         "平均值MAE": mae_mean,
243         "改进百分比": improvement,
244         "模型类型": "线性回归"
245     })
246
247     # 存储方程
248     equations.append({
249         "BMI分组": bmi_cat,
250         "方程": equation,
251         "模型类型": "线性回归",
252         "最优时点": optimal_time
253     })
254
255     except Exception as e:
256         print(f"线性回归建模出错: {e}")
257         age_importance = np.nan
258         height_importance = np.nan
259         weight_importance = np.nan
260         model_r2 = np.nan
261         model_mae = np.nan
262         equation = "无法生成方程"
263
264     try:
265         # 标准化特征
266         scaler = StandardScaler()
267         X_scaled = scaler.fit_transform(X)
268
269         # 使用随机森林回归
270         rf = RandomForestRegressor(n_estimators=100, random_state=42)
271
272         # 交叉验证评估模型性能

```



```

273 kf = KFold(n_splits=min(5, len(X)), shuffle=True, random_state=42)
274 r2_scores = cross_val_score(rf, X_scaled, y, cv=kf, scoring='r2')
275 mae_scores = -cross_val_score(rf, X_scaled, y, cv=kf, scoring='neg_mean_absolute_error')
276
277 # 训练最终模型
278 rf.fit(X_scaled, y)
279
280 # 获取特征重要性
281 feature_importance = rf.feature_importances_
282 age_importance = feature_importance[0]
283 height_importance = feature_importance[1]
284 weight_importance = feature_importance[2]
285
286 # 计算模型性能
287 model_r2 = np.mean(r2_scores)
288 model_mae = np.mean(mae_scores)
289
290 print(f"随机森林模型性能:")
291 print(f" - R²得分: {model_r2:.4f}")
292 print(f" - 平均绝对误差: {model_mae:.2f}天")
293 print(f"特征重要性:")
294 print(f" - 年龄: {age_importance:.4f}")
295 print(f" - 身高: {height_importance:.4f}")
296 print(f" - 体重: {weight_importance:.4f}")
297
298 # 使用SHAP值解释模型
299 if len(X) >= 20: # 只有当样本量足够时才计算SHAP值
300     try:
301         explainer = shap.TreeExplainer(rf)
302         shap_values = explainer.shap_values(X_scaled)
303
304         plt.figure(figsize=(10, 6))
305         shap.summary_plot(shap_values, X_scaled, feature_names=['年龄', '身高', '体重'],
306                           show=False)
307         plt.title(f'BMI分组 {bmi_cat} 特征影响分析 (SHAP值)')
308         plt.tight_layout()
309         plt.savefig(f'./python_code/BMI-{bmi_cat.replace("<", "lt").replace(" ", "ge")}.
310                     replace(" ", "_").replace("(", "").replace(")", "")}_shap.png', dpi=300,
311                     bbox_inches="tight")
312         plt.close()
313         print("SHAP分析图表已保存")
314     except Exception as e:
315         print(f"SHAP分析出错: {e}")
316
317 # 验证引入影响因素后的改进
318 y_pred = rf.predict(X_scaled)
319 y_mean = np.full_like(y_pred, y.mean())
320 mae_model = np.mean(np.abs(y - y_pred))
321 mae_mean = np.mean(np.abs(y - y_mean))
322 improvement = (mae_mean - mae_model) / mae_mean * 100
323
324 print(f"\n模型验证结果:")
325 print(f" - 仅使用平均值预测的MAE: {mae_mean:.2f}天")
326 print(f" - 使用完整模型预测的MAE: {mae_model:.2f}天")
327 print(f" - 改进百分比: {improvement:.2f}%")

```

```

325     model_performance.append({
326         "BMI 分组": bmi_cat,
327         "样本量": len(X),
328         "R² 得分": model_r2,
329         "模型MAE": mae_model,
330         "平均值MAE": mae_mean,
331         "改进百分比": improvement,
332         "模型类型": "随机森林"
333     })
334
335
336     # 为随机森林生成更合理的方程描述
337     # 计算每个特征的相对影响
338     age_effect = age_importance * 100
339     height_effect = height_importance * 100
340     weight_effect = weight_importance * 100
341
342     # 生成方程描述
343     equation = f"T    {optimal_time:.1f} + {age_effect:.1f}%×(年龄影响) + {height_effect:.1f}%×(身高影响) + {weight_effect:.1f}%×(体重影响)"
344
345     # 存储方程
346     equations.append({
347         "BMI 分组": bmi_cat,
348         "方程": equation,
349         "模型类型": "随机森林",
350         "最优时点": optimal_time
351     })
352
353     except Exception as e:
354         print(f"随机森林建模出错: {e}")
355         age_importance = np.nan
356         height_importance = np.nan
357         weight_importance = np.nan
358         model_r2 = np.nan
359         model_mae = np.nan
360         equation = "无法生成方程"
361
362     # 可视化影响因素
363     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
364
365     # 1. 生存曲线和效用函数
366     ax = axes[0, 0]
367     (1 - kmf.survival_function_).plot(
368         ax=ax,
369         label=f"累积达标函数 F(t) ({bmi_cat})",
370         color="seagreen",
371         linewidth=2.5,
372     )
373
374     time_points = np.linspace(0, df_bmi["达标时间"].max(), 200)
375     utilities = [calculate_utility(t, s_t) for t in time_points]
376     ax_twin = ax.twinx()
377     ax_twin.plot(
378         time_points, utilities, "--", color="red", label="效用函数 E(t)", alpha=0.6

```

```

379 )
380
381 ax_twin.axvline(x=optimal_time, color='purple', linestyle='--', linewidth=2)
382 ax_twin.text(
383     optimal_time + 2,
384     min(utilities) + 0.1,
385     f"最优时间点: {optimal_time:.1f}天",
386     color='purple',
387     fontsize=10,
388 )
389
390 ax.axvspan(0, 84, facecolor='green', alpha=0.15, label='早期阶段 (<12周)')
391 ax.axvspan(84, 189, facecolor='orange', alpha=0.15, label='中期阶段 (12-27周)')
392 ax.axvspan(
393     189,
394     df_bmi["达标时间"].max() + 10,
395     facecolor='red',
396     alpha=0.15,
397     label='晚期阶段 (>27周)',
398 )
399
400 ax.set_title(f"BMI分组 {bmi_cat} 达标情况与效用分析", fontsize=14)
401 ax.set_xlabel("天数", fontsize=12)
402 ax.set_ylabel("累积达标比例", fontsize=12)
403 ax_twin.set_ylabel("效用值", fontsize=12)
404
405 lines1, labels1 = ax.get_legend_handles_labels()
406 lines2, labels2 = ax_twin.get_legend_handles_labels()
407 ax.legend(lines1 + lines2, labels1 + labels2, loc='upper left')
408
409 ax.grid(True, alpha=0.3)
410 ax.set_ylim(0, 1.05)
411 ax_twin.set_ylim(0, max(utilities) * 1.2)
412 ax.set_xlim(-5, df_bmi["达标时间"].max() + 10)
413
414 # 2. 年龄与达标时间的关系
415 axes[0, 1].scatter(df_bmi['年龄'], df_bmi['达标时间'], alpha=0.6)
416 if not np.isnan(age_importance):
417     # 使用局部加权散点平滑 (LOWESS) 拟合曲线
418     try:
419         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['年龄'], frac=0.7)
420         axes[0, 1].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r-', alpha=0.8, label='LOWESS 拟合')
421     except:
422         # 如果LOWESS失败, 使用多项式拟合
423         z = np.polyfit(df_bmi['年龄'], df_bmi['达标时间'], 2)
424         p = np.poly1d(z)
425         x_sorted = np.sort(df_bmi['年龄'])
426         axes[0, 1].plot(x_sorted, p(x_sorted), 'r-', alpha=0.8, label='多项式拟合')
427 axes[0, 1].set_xlabel('年龄')
428 axes[0, 1].set_ylabel('达标时间(天)')
429 axes[0, 1].set_title('年龄与达标时间的关系')
430 axes[0, 1].grid(True, alpha=0.3)
431 if not np.isnan(age_importance):
432     axes[0, 1].text(0.05, 0.95, f'重要性: {age_importance:.4f}', transform=axes[0, 1].transAxes,
433                    fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='

```

```

wheat', alpha=0.5))

434
435 # 3. 身高与达标时间的关系
436 axes[1, 0].scatter(df_bmi['身高'], df_bmi['达标时间'], alpha=0.6)
437 if not np.isnan(height_importance):
438     try:
439         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['身高'], frac=0.7)
440         axes[1, 0].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r--', alpha=0.8, label='LOWESS 拟合')
441     except:
442         z = np.polyfit(df_bmi['身高'], df_bmi['达标时间'], 2)
443         p = np.poly1d(z)
444         x_sorted = np.sort(df_bmi['身高'])
445         axes[1, 0].plot(x_sorted, p(x_sorted), 'r--', alpha=0.8, label='多项式拟合')
446 axes[1, 0].set_xlabel('身高(cm)')
447 axes[1, 0].set_ylabel('达标时间(天)')
448 axes[1, 0].set_title('身高与达标时间的关系')
449 axes[1, 0].grid(True, alpha=0.3)
450 if not np.isnan(height_importance):
451     axes[1, 0].text(0.05, 0.95, f'重要性: {height_importance:.4f}', transform=axes[1, 0].
452         transAxes,
453         fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='
454             wheat', alpha=0.5))
455
456 # 4. 体重与达标时间的关系
457 axes[1, 1].scatter(df_bmi['体重'], df_bmi['达标时间'], alpha=0.6)
458 if not np.isnan(weight_importance):
459     try:
460         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['体重'], frac=0.7)
461         axes[1, 1].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r--', alpha=0.8, label='LOWESS 拟合')
462     except:
463         z = np.polyfit(df_bmi['体重'], df_bmi['达标时间'], 2)
464         p = np.poly1d(z)
465         x_sorted = np.sort(df_bmi['体重'])
466         axes[1, 1].plot(x_sorted, p(x_sorted), 'r--', alpha=0.8, label='多项式拟合')
467 axes[1, 1].set_xlabel('体重(kg)')
468 axes[1, 1].set_ylabel('达标时间(天)')
469 axes[1, 1].set_title('体重与达标时间的关系')
470 axes[1, 1].grid(True, alpha=0.3)
471 if not np.isnan(weight_importance):
472     axes[1, 1].text(0.05, 0.95, f'重要性: {weight_importance:.4f}', transform=axes[1, 1].
473         transAxes,
474         fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='
475             wheat', alpha=0.5))
476
477 plt.tight_layout()
478
479 # 保存图表
480 filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(" ", "ge").replace(" ", "_").
481     replace("<", "").replace(">", "")}_analysis.png'
482 plt.savefig(filename, dpi=300, bbox_inches="tight")
483 plt.close()
484
485 print(f"\n图表已保存至: {filename}")
486
487 results_table.append({

```

```

483         "BMI分组": bmi_cat,
484         "最优时点(天)": f"{optimal_time:.1f}",
485         "最优时点(周)": f"{optimal_time/7:.1f}",
486         "最小效用值": f"{optimal_utility:.4f}",
487         "风险水平": f"{risk_level:.4f}",
488         "样本量": sample_size,
489         "年龄重要性": f"{age_importance:.4f}" if not np.isnan(age_importance) else "N/A",
490         "身高重要性": f"{height_importance:.4f}" if not np.isnan(height_importance) else "N/A",
491         "体重重要性": f"{weight_importance:.4f}" if not np.isnan(weight_importance) else "N/A",
492         "模型R²": f"{model_r2:.4f}" if not np.isnan(model_r2) else "N/A",
493         "模型MAE": f"{model_mae:.2f}" if not np.isnan(model_mae) else "N/A",
494     })
495
496     print("\n" + "="*60 + "\n")
497
498     # 打印和保存结果表格
499     print("\n\n=== 各BMI分组NIPT时点计算结果 ===")
500     results_df = pd.DataFrame(results_table)
501     print(results_df.to_string(index=False))
502
503     results_df.to_excel("./python_code/NIPT_optimal_times_with_advanced_modeling.xlsx", index=False)
504     print(f"\n结果表格已保存至: ./python_code/NIPT_optimal_times_with_advanced_modeling.xlsx")
505
506     # 保存模型性能评估结果
507     if model_performance:
508         performance_df = pd.DataFrame(model_performance)
509         performance_df.to_excel("./python_code/model_performance_evaluation.xlsx", index=False)
510         print(f"模型性能评估已保存至: ./python_code/model_performance_evaluation.xlsx")
511
512     # 绘制模型性能比较图
513     plt.figure(figsize=(10, 6))
514     x = np.arange(len(performance_df))
515     width = 0.35
516
517     plt.bar(x - width/2, performance_df['平均值MAE'], width, label='仅使用平均值')
518     plt.bar(x + width/2, performance_df['模型MAE'], width, label='使用完整模型')
519
520     plt.xlabel('BMI分组')
521     plt.ylabel('平均绝对误差(天)')
522     plt.title('模型性能比较: 仅使用平均值 vs 使用完整模型')
523     plt.xticks(x, performance_df['BMI分组'])
524     plt.legend()
525     plt.grid(True, alpha=0.3)
526
527     # 在柱状图上添加数值标签
528     for i, v in enumerate(performance_df['平均值MAE']):
529         plt.text(i - width/2, v + 0.5, f'{v:.1f}', ha='center')
530
531     for i, v in enumerate(performance_df['模型MAE']):
532         plt.text(i + width/2, v + 0.5, f'{v:.1f}', ha='center')
533
534     plt.tight_layout()
535     plt.savefig('./python_code/model_performance_comparison.png', dpi=300, bbox_inches='tight')
536     plt.close()
537     print("模型性能比较图已保存")

```

```

538
539 # 创建更合理的NIPT时点方程
540 print("\n\n=== 各BMI分组NIPT时点方程 ===")
541 improved_equations = []
542
543 for bmi_cat in bmi_categories:
544     if bmi_cat not in optimal_times:
545         improved_equations.append({
546             "BMI分组": bmi_cat,
547             "NIPT时点方程": "无数据",
548             "模型类型": "无数据"
549         })
550         continue
551
552     optimal_time = optimal_times[bmi_cat]
553
554     # 根据BMI分组提供个性化的NIPT时点建议
555     if bmi_cat == "聚类0 (<30.01)":
556         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI<30.01的
                    孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
557     elif bmi_cat == "聚类1 (30.01-32.18)":
558         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    30.01-32.18的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
559     elif bmi_cat == "聚类2 (32.18-34.63)":
560         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    32.18-34.63的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
561     elif bmi_cat == "聚类3 (34.63-37.93)":
562         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    34.63-37.93的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
563     else: # 聚类4 (37.93)
564         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI 37.93的
                    孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
565
566     # 确定模型类型
567     model_type = "线性回归" if bmi_cat == "聚类4 (37.93)" else "随机森林"
568
569     improved_equations.append({
570         "BMI分组": bmi_cat,
571         "NIPT时点方程": equation,
572         "模型类型": model_type
573     })
574
575     print(f"\n{bmi_cat}:")
576     print(equation)
577
578 # 保存改进的NIPT时点方程
579 improved_equations_df = pd.DataFrame(improved_equations)
580 improved_equations_df.to_excel("./python_code/improved_NIPT_time_equations.xlsx", index=False)
581 print(f"\n改进的NIPT时点方程已保存至: ./python_code/improved_NIPT_time_equations.xlsx")
582
583 # 创建影响因素汇总图
584 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
585
586 # 提取影响系数
587 age_effects = []

```

```

588 height_effects = []
589 weight_effects = []
590 labels = []
591
592 for _, row in results_df.iterrows():
593     if row['年龄重要性'] != 'N/A':
594         age_effects.append(float(row['年龄重要性']))
595         height_effects.append(float(row['身高重要性']))
596         weight_effects.append(float(row['体重重要性']))
597         labels.append(row['BMI分组'].split(' ')[0])
598
599 # 绘制年龄影响
600 if age_effects:
601     axes[0].bar(range(len(age_effects)), age_effects, color='skyblue')
602     axes[0].set_xlabel('BMI分组')
603     axes[0].set_ylabel('特征重要性')
604     axes[0].set_title('年龄对达标时间的影响')
605     axes[0].set_xticks(range(len(age_effects)))
606     axes[0].set_xticklabels(labels)
607
608 # 绘制身高影响
609 if height_effects:
610     axes[1].bar(range(len(height_effects)), height_effects, color='lightgreen')
611     axes[1].set_xlabel('BMI分组')
612     axes[1].set_ylabel('特征重要性')
613     axes[1].set_title('身高对达标时间的影响')
614     axes[1].set_xticks(range(len(height_effects)))
615     axes[1].set_xticklabels(labels)
616
617 # 绘制体重影响
618 if weight_effects:
619     axes[2].bar(range(len(weight_effects)), weight_effects, color='lightcoral')
620     axes[2].set_xlabel('BMI分组')
621     axes[2].set_ylabel('特征重要性')
622     axes[2].set_title('体重对达标时间的影响')
623     axes[2].set_xticks(range(len(weight_effects)))
624     axes[2].set_xticklabels(labels)
625
626 plt.tight_layout()
627 plt.savefig('./python_code/factors_impact_summary_advanced.png', dpi=300, bbox_inches='tight')
628 plt.close()
629
630 print("\n影响因素汇总图已保存至: ./python_code/factors_impact_summary_advanced.png")
631
632 error_analysis_results = analyze_error_impact(survival_df, df, bmi_categories, num_simulations=100,
633                                              error_std=0.01)
634
635 return results_df, improved_equations_df, error_analysis_results
636
637 # 分析检测误差对最优时点、效用值和风险的影响
638 def analyze_error_impact(survival_df, original_df, bmi_categories, num_simulations=100, error_std=0.01):
639     # 辅助函数: 清理文件名中的非法字符
640     def clean_filename(name):
641         return re.sub(r'[\<>:"/\\|?*]', "_", name)

```

```

642 # 存储原始最优时点
643 original_optimal_times = {}
644 error_results = {bmi_cat: {"times": [], "utilities": [], "risks": []} for bmi_cat in bmi_categories}
645
646 # 定义效用函数（与主函数中相同）
647 def get_rearly(t):
648     return 2.0 * np.exp(-t / 50)
649
650 def get_rlate(t):
651     if t < 84:
652         return 0.1
653     elif t <= 189:
654         normalized_t = (t - 84) / (189 - 84)
655         return 0.1 + 0.9 * (normalized_t**2)
656     else:
657         return 1.0
658
659 def calculate_utility(t, survival_func):
660     if t < 0:
661         return float("inf")
662
663     closest_time_idx = np.abs(survival_func.index - t).argmin()
664     actual_time = survival_func.index[closest_time_idx]
665     p_t = 1 - survival_func.loc[actual_time, "s_t"]
666     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
667
668 # 计算原始最优时点
669 for bmi_cat in bmi_categories:
670     df_bmi = survival_df[survival_df['bmi_category'] == bmi_cat].copy()
671     if df_bmi.empty:
672         continue
673
674     kmf = KaplanMeierFitter()
675     kmf.fit(durations=df_bmi["达标时间"], event_observed=df_bmi["达标状态"])
676     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
677
678     def objective(t):
679         return calculate_utility(t, s_t)
680
681     result = minimize_scalar(objective, bounds=(0, df_bmi["达标时间"].max()), method="bounded")
682     original_optimal_times[bmi_cat] = result.x
683
684 # 开始误差模拟
685 for sim in range(num_simulations):
686     print(f"正在进行第 {sim+1} 次误差模拟...")
687     np.random.seed(sim) # 确保可重复性
688
689     # 创建带误差的数据副本
690     df_error = original_df.copy()
691
692     # 对Y染色体浓度添加随机误差
693     df_error['Y染色体浓度_error'] = df_error['Y染色体浓度'] + np.random.normal(
694         0, error_std, size=len(df_error))
695
696

```



```

697 # 重新计算每个孕妇的达标时间（带误差）
698 survival_data_error = []
699
700 for code, group in df_error.groupby('孕妇代码'):
701     group = group.sort_values('检测孕周_天数')
702
703     # 找到首次达标的时间（使用带误差的Y染色体浓度）
704     first_reach_idx = group[group['Y染色体浓度_error'] >= 0.04].index
705     last_below_idx = group[group['Y染色体浓度_error'] < 0.04].index
706
707     if len(first_reach_idx) > 0:
708         first_reach_time = group.loc[first_reach_idx[0], '检测孕周_天数']
709         first_reach_concentration = group.loc[first_reach_idx[0], 'Y染色体浓度_error']
710
711         if len(last_below_idx) > 0 and last_below_idx[-1] < first_reach_idx[0]:
712             last_below_time = group.loc[last_below_idx[-1], '检测孕周_天数']
713             last_below_concentration = group.loc[last_below_idx[-1], 'Y染色体浓度_error']
714
715             # 使用线性插值法计算达标时间
716             event_time = last_below_time + (first_reach_time - last_below_time) * \
717                 (0.04 - last_below_concentration) / (first_reach_concentration -
718                 last_below_concentration)
719
720             else:
721                 event_time = first_reach_time
722
723             event_observed = 1
724         else:
725             event_time = group.iloc[-1]['检测孕周_天数']
726             event_observed = 0
727
728     base_info = group.iloc[0]
729
730     survival_data_error.append({
731         '孕妇代码': code,
732         '年龄': base_info['年龄'],
733         '身高': base_info['身高'],
734         '体重': base_info['体重'],
735         '孕妇BMI': base_info['孕妇BMI'],
736         'bmi_category': base_info['bmi_category'],
737         '达标时间': event_time,
738         '达标状态': event_observed
739     })
740
741     survival_df_error = pd.DataFrame(survival_data_error)
742
743     # 对每个BMI分组计算带误差的最优时点
744     for bmi_cat in bmi_categories:
745         df_bmi_error = survival_df_error[survival_df_error['bmi_category'] == bmi_cat].copy()
746         if df_bmi_error.empty:
747             continue
748
749         kmf_error = KaplanMeierFitter()
750         kmf_error.fit(durations=df_bmi_error["达标时间"], event_observed=df_bmi_error["达标状态"])
751         s_t_error = kmf_error.survival_function_.rename(columns={"KM_estimate": "s_t"})

```

```

751     def objective_error(t):
752         return calculate_utility(t, s_t_error)
753
754     result_error = minimize_scalar(
755         objective_error, bounds=(0, df_bmi_error["达标时间"].max()), method="bounded"
756     )
757
758     optimal_time_error = result_error.x
759     optimal_utility_error = result_error.fun
760     risk_error = 1 / optimal_utility_error if optimal_utility_error > 0 else float("inf")
761
762     error_results[bmi_cat]["times"].append(optimal_time_error)
763     error_results[bmi_cat]["utilities"].append(optimal_utility_error)
764     error_results[bmi_cat]["risks"].append(risk_error)
765
766 # 输出误差分析结果
767 print("\n=== 检测误差影响分析结果 ===")
768 error_analysis_summary = []
769
770 for bmi_cat in bmi_categories:
771     if error_results[bmi_cat]["times"]:
772         times = np.array(error_results[bmi_cat]["times"])
773         utilities = np.array(error_results[bmi_cat]["utilities"])
774         risks = np.array(error_results[bmi_cat]["risks"])
775
776         mean_time = times.mean()
777         std_time = times.std()
778         time_ci = norm.interval(0.95, loc=mean_time, scale=std_time/np.sqrt(len(times)))
779
780         mean_utility = utilities.mean()
781         std_utility = utilities.std()
782
783         mean_risk = risks.mean()
784         std_risk = risks.std()
785
786         original_time = original_optimal_times.get(bmi_cat, 0)
787         original_utility = calculate_utility(original_time, s_t) if bmi_cat in
788             original_optimal_times else 0
789
790         print(f"BMI分组 {bmi_cat}:")
791         print(f"  原始最优时点: {original_time:.2f} 天")
792         print(f"  模拟最优时点均值: {mean_time:.2f} 天")
793         print(f"  标准差: {std_time:.2f} 天")
794         print(f"  95%置信区间: ({time_ci[0]:.2f}, {time_ci[1]:.2f}) 天")
795         print(f"  原始最小效用值: {original_utility:.4f}")
796         print(f"  模拟最小效用值均值: {mean_utility:.4f}")
797         print(f"  效用值标准差: {std_utility:.4f}")
798         print(f"  模拟风险水平均值: {mean_risk:.4f}")
799         print(f"  风险水平标准差: {std_risk:.4f}")
800
801 # 可视化误差分析结果
802 plt.figure(figsize=(12, 8))
803
804 plt.subplot(2, 2, 1)
805 plt.hist(times, bins=20, alpha=0.7, edgecolor="black")

```

```

805 plt.axvline(mean_time, color="r", linestyle="--", label=f"模拟平均时点: {mean_time:.2f}")
806 plt.axvline(original_time, color="b", linestyle="-", label=f"原始最优时点: {original_time:.2f}")
807 plt.axvline(time_ci[0], color="g", linestyle=":", label="95%置信区间")
808 plt.axvline(time_ci[1], color="g", linestyle=":")
809 plt.xlabel("最优NIPT时点 (天)")
810 plt.ylabel("频数")
811 plt.title(f"BMI分组 {bmi_cat} - 最优时点分布")
812 plt.legend()
813 plt.grid(True, alpha=0.3)
814
815 plt.subplot(2, 2, 2)
816 plt.hist(utilities, bins=20, alpha=0.7, edgecolor="black", color="orange")
817 plt.axvline(mean_utility, color="r", linestyle="--", label=f"模拟平均效用值: {mean_utility:.4f}")
818 plt.axvline(original_utility, color="b", linestyle="-", label=f"原始最小效用值: {original_utility:.4f}")
819 plt.xlabel("最小效用值")
820 plt.ylabel("频数")
821 plt.title(f"BMI分组 {bmi_cat} - 最小效用值分布")
822 plt.legend()
823 plt.grid(True, alpha=0.3)
824
825 plt.subplot(2, 2, 3)
826 plt.scatter(times, utilities, alpha=0.6)
827 plt.axvline(original_time, color="b", linestyle="-", label="原始最优时点")
828 plt.axhline(original_utility, color="b", linestyle="-", label="原始最小效用值")
829 plt.xlabel("最优时点 (天)")
830 plt.ylabel("最小效用值")
831 plt.title(f"BMI分组 {bmi_cat} - 时点与效用值关系")
832 plt.legend()
833 plt.grid(True, alpha=0.3)
834
835 plt.subplot(2, 2, 4)
836 plt.hist(risks, bins=20, alpha=0.7, edgecolor="black", color="red")
837 plt.axvline(mean_risk, color="r", linestyle="--", label=f"模拟平均风险: {mean_risk:.4f}")
838 plt.xlabel("风险水平")
839 plt.ylabel("频数")
840 plt.title(f"BMI分组 {bmi_cat} - 风险水平分布")
841 plt.legend()
842 plt.grid(True, alpha=0.3)
843
844 plt.tight_layout()
845
846 safe_bmi_cat = clean_filename(bmi_cat)
847 filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.png"
848 plt.savefig(filename, dpi=300, bbox_inches="tight")
849 plt.close()
850
851 # 保存模拟数据
852 data_df = pd.DataFrame({
853     "time": times,
854     "utility": utilities,
855     "risk": risks
856 })

```

```

857     csv_filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.csv"
858     data_df.to_csv(csv_filename, index=False)
859
860     print(f" 图表已保存至: {filename}")
861     print(f" 数据已保存至: {csv_filename}")
862
863     error_analysis_summary.append({
864         "BMI分组": bmi_cat,
865         "原始最优时点": original_time,
866         "模拟最优时点均值": mean_time,
867         "最优时点标准差": std_time,
868         "最优时点95%置信区间下限": time_ci[0],
869         "最优时点95%置信区间上限": time_ci[1],
870         "原始最小效用值": original_utility,
871         "模拟最小效用值均值": mean_utility,
872         "最小效用值标准差": std_utility,
873         "模拟风险水平均值": mean_risk,
874         "风险水平标准差": std_risk
875     })
876     else:
877         print(f"BMI分组 {bmi_cat}: 无数据")
878         error_analysis_summary.append({
879             "BMI分组": bmi_cat,
880             "原始最优时点": "无数据",
881             "模拟最优时点均值": "无数据",
882             "最优时点标准差": "无数据",
883             "最优时点95%置信区间下限": "无数据",
884             "最优时点95%置信区间上限": "无数据",
885             "原始最小效用值": "无数据",
886             "模拟最小效用值均值": "无数据",
887             "最小效用值标准差": "无数据",
888             "模拟风险水平均值": "无数据",
889             "风险水平标准差": "无数据"
890         })
891
892     # 保存误差分析摘要
893     error_summary_df = pd.DataFrame(error_analysis_summary)
894     error_summary_df.to_excel("./python_code/error_analysis_summary.xlsx", index=False)
895     print(f"\n误差分析摘要已保存至: ./python_code/error_analysis_summary.xlsx")
896
897     return error_summary_df
898
899 results, equations, error_results = analyze_nipt_optimal_time_with_advanced_modeling()

```

Listing 9: 数据处理.py

```

1  import pandas as pd
2  import numpy as np
3
4
5  def process_female_nipt_data(input_file_path, output_file_path):
6      """
7      处理女胎NIPT数据，进行质量控制和数据清洗，然后保存到新的Excel文件
8
9      参数：

```

```

10 input_file_path: 输入Excel文件路径
11 output_file_path: 输出Excel文件路径
12 """
13
14 # 1. 读取数据 - 指定读取第二个工作表
15 print("正在读取数据...")
16 try:
17     # 读取第二个工作表
18     df = pd.read_excel(input_file_path, sheet_name=1)
19     print(f"成功读取数据: {df.shape[0]} 行, {df.shape[1]} 列")
20 except Exception as e:
21     print(f"读取数据时出错: {e}")
22     return
23
24 # 2. 添加三种疾病的one-hot编码列
25 print("\n正在添加疾病分类列...")
26
27 # 确保"染色体的非整倍体"列存在
28 if "染色体的非整倍体" not in df.columns:
29     print("错误: 未找到'染色体的非整倍体'列")
30     return
31
32 # 创建三种疾病的标志列
33 df["T21"] = 0 # 唐氏综合征 (21号染色体三体)
34 df["T18"] = 0 # 爱德华氏综合征 (18号染色体三体)
35 df["T13"] = 0 # 帕陶氏综合征 (13号染色体三体)
36
37 # 根据"染色体的非整倍体"列设置疾病标志
38 for idx, value in df["染色体的非整倍体"].items():
39     if pd.isna(value) or value == "":
40         continue
41
42     # 根据内容设置相应的疾病标志
43     if "T21" in str(value) or "21" in str(value):
44         df.at[idx, "T21"] = 1
45     if "T18" in str(value) or "18" in str(value):
46         df.at[idx, "T18"] = 1
47     if "T13" in str(value) or "13" in str(value):
48         df.at[idx, "T13"] = 1
49
50 # 统计疾病分布
51 t21_count = df["T21"].sum()
52 t18_count = df["T18"].sum()
53 t13_count = df["T13"].sum()
54 normal_count = len(df) - (t21_count + t18_count + t13_count)
55
56 print(f"正常样本数: {normal_count}")
57 print(f"T21 (唐氏综合征) 样本数: {t21_count}")
58 print(f"T18 (爱德华氏综合征) 样本数: {t18_count}")
59 print(f"T13 (帕陶氏综合征) 样本数: {t13_count}")
60
61 # 3. 数据质量检查
62 print("\n正在进行数据质量检查...")
63
64 # 检查缺失值

```

```

65 missing_values = df.isnull().sum()
66 if missing_values.sum() > 0:
67     print("发现缺失值:")
68     print(missing_values[missing_values > 0])
69 else:
70     print("未发现缺失值")
71
72 # 检查重复行
73 duplicates = df.duplicated().sum()
74 if duplicates > 0:
75     print(f"发现 {duplicates} 个重复行, 已移除")
76     df = df.drop_duplicates()
77
78 # 4. 数据质量控制 - 专门针对女胎数据
79 print("\n正在进行数据质量控制...")
80 initial_count = len(df)
81
82 # 应用质量控制阈值
83 qc_conditions = (
84     (df["原始读段数"] > 3500000)
85     & (df["在参考基因组上比对的比例"] > 0.75)
86     & (df["重复读段的比例"] < 0.35)
87     & (df["唯一比对的读段数"] / df["原始读段数"] > 0.7)
88     & (df["被过滤掉读段数的比例"] < 0.5)
89     &
90     # 对于女胎, X染色体浓度可能为负值, 但其他质量指标正常即可
91     (df["GC含量"].between(0.39, 0.6)) # GC含量应在正常范围内
92     & (df["X染色体的Z值"] <= 3)
93     & (df["X染色体的Z值"] >= -3)
94 )
95
96 # 筛选高质量数据
97 hq_df = df[qc_conditions].copy()
98 filtered_count = initial_count - len(hq_df)
99
100 print(f"初始样本数: {initial_count}")
101 print(f"高质量样本数: {len(hq_df)}")
102 print(f"被过滤样本数: {filtered_count}")
103 print(f"过滤比例: {filtered_count/initial_count*100:.2f}%")
104
105 # 5. 保存处理后的数据
106 print("\n正在保存处理后的数据...")
107 try:
108     hq_df.to_excel(output_file_path, index=False)
109     print(f"处理后的数据已保存到: {output_file_path}")
110     print("处理完成!")
111
112 except Exception as e:
113     print(f"保存数据时出错: {e}")
114
115 # 使用示例
116 if __name__ == "__main__":
117     # 请修改为您的实际文件路径
118     input_file = "./python_code/附件.xlsx"

```

```

120     output_file = "./python_code/Q4/处理后的女胎NIPT数据.xlsx"
121
122     process_female_nipt_data(input_file, output_file)
123
124     # 初始样本数: 605
125     # 高质量样本数: 556
126     # 被过滤样本数: 49
127     # 过滤比例: 8.10%

```

Listing 10: 点二列相关系数 _smote_ 可视化 _final.py

```

1  import pandas as pd
2  import numpy as np
3  from scipy.stats import pointbiserialr
4  from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold,
   RandomizedSearchCV
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.compose import ColumnTransformer
7  from sklearn.pipeline import Pipeline
8  from sklearn.ensemble import RandomForestClassifier
9  from xgboost import XGBClassifier
10 from sklearn.svm import SVC
11 from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, f1_score,
   make_scorer, accuracy_score, precision_score, recall_score, roc_curve, precision_recall_curve
12 from imblearn.over_sampling import SMOTE, ADASYN
13 from imblearn.combine import SMOTEENN
14 from collections import Counter
15 import matplotlib.pyplot as plt
16 import seaborn as sns
17 import warnings
18 warnings.filterwarnings('ignore')
19
20 # 设置中文字体
21 plt.rcParams['font.sans-serif'] = ['SimHei']
22 plt.rcParams['axes.unicode_minus'] = False
23
24 # 设置颜色方案
25 colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd', '#8c564b', '#e377c2', '#7f7f7f', '#
   bcbd22', '#17becf']
26
27 # 加载和预处理数据函数保持不变
28 def load_and_preprocess_data(file_path):
29     """加载并预处理数据"""
30     print("正在加载数据...")
31
32     try:
33         # 直接读取Excel文件
34         df = pd.read_excel(file_path)
35         print(f"成功读取数据, 形状: {df.shape}")
36
37     except Exception as e:
38         print(f"读取Excel文件时出错: {e}")
39         return None
40
41     # 处理缺失值

```

```

42     print("\n处理缺失值...")
43
44     # 删除Unnamed列 (如果存在)
45     columns_to_drop = [col for col in df.columns if 'Unnamed' in str(col)]
46     if columns_to_drop:
47         df = df.drop(columns=columns_to_drop)
48         print(f"已删除列: {columns_to_drop}")
49
50     # 处理孕妇BMI缺失值
51     if '孕妇BMI' in df.columns and df['孕妇BMI'].isnull().sum() > 0:
52         bmi_median = df['孕妇BMI'].median()
53         df['孕妇BMI'] = df['孕妇BMI'].fillna(bmi_median)
54         print(f"使用中位数 {bmi_median:.2f} 填充孕妇BMI缺失值")
55
56     # 创建目标变量 - 是否有任何染色体异常
57     if all(col in df.columns for col in ['T21', 'T18', 'T13']):
58         df['染色体异常'] = df[['T21', 'T18', 'T13']].max(axis=1)
59         print("使用现有的T21, T18, T13列创建目标变量")
60     elif '染色体的非整倍体' in df.columns:
61         # 如果没有T21, T18, T13列, 但有染色体的非整倍体列, 则创建它们
62         df['T21'] = 0
63         df['T18'] = 0
64         df['T13'] = 0
65
66         for idx, value in df['染色体的非整倍体'].items():
67             if pd.isna(value) or value == '':
68                 continue
69
70             value_str = str(value).upper()
71             if 'T21' in value_str or '21' in value_str:
72                 df.at[idx, 'T21'] = 1
73             if 'T18' in value_str or '18' in value_str:
74                 df.at[idx, 'T18'] = 1
75             if 'T13' in value_str or '13' in value_str:
76                 df.at[idx, 'T13'] = 1
77
78         df['染色体异常'] = df[['T21', 'T18', 'T13']].max(axis=1)
79         print("使用'染色体的非整倍体'列创建目标变量")
80     else:
81         print("错误: 无法找到创建目标变量所需的列")
82         return None
83
84     print(f"\n目标变量分布:")
85     print(df['染色体异常'].value_counts())
86     print(f"异常样本比例: {df['染色体异常'].mean()*100:.2f}%")
87
88     return df
89
90 # 准备特征函数保持不变
91 def prepare_features(df):
92     """准备特征矩阵和目标向量"""
93     # 可能的特征列 - 只使用数值型特征
94     possible_features = [
95         '孕妇年龄', '孕妇BMI', '怀孕次数', '生产次数', # 基本信息
96         'X染色体的Z值', '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值', # Z值

```



```

97         'GC含量', '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量', # GC相关
98         '原始读段数', '唯一比对的读段数', '重复读段的比例', '在参考基因组上比对的比例', '被过滤掉读段数
          的比例' # 测序质量
99     ]
100
101     # 只保留实际存在且为数值型的列
102     available_features = []
103     for col in possible_features:
104         if col in df.columns:
105             # 检查是否为数值型
106             if pd.api.types.is_numeric_dtype(df[col]):
107                 available_features.append(col)
108             else:
109                 print(f"警告: 列 {col} 不是数值型, 将被忽略")
110
111     print(f"\n使用的数值型特征: {available_features}")
112
113     if not available_features:
114         print("错误: 没有找到任何数值型特征列")
115         return None, None
116
117     X = df[available_features]
118     y = df['染色体异常']
119
120     # 确保所有特征都是数值型
121     for col in X.columns:
122         if not pd.api.types.is_numeric_dtype(X[col]):
123             print(f"错误: 列 {col} 不是数值型, 但被包含在特征中")
124             return None, None
125
126     return X, y
127
128 # 点二列相关系数筛选函数保持不变
129 def filter_features_point_biserial(X, y, threshold=0.1, top_k=None):
130     """
131     使用点二列相关系数进行初步特征筛选
132     """
133     print("\n=== 使用点二列相关系数进行初步特征筛选 ===")
134
135     # 标准化特征 (确保公平比较)
136     scaler = StandardScaler()
137     X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns, index=X.index)
138
139     correlations = []
140     p_values = []
141
142     # 计算每个特征与目标变量的点二列相关系数
143     for feature in X_scaled.columns:
144         # 移除缺失值
145         mask = ~(X_scaled[feature].isnull() | y.isnull())
146         x_vals = X_scaled.loc[mask, feature]
147         y_vals = y.loc[mask]
148
149         # 计算点二列相关系数
150         corr, p_val = pointbiserialr(x_vals, y_vals)

```

```

151     correlations.append(corr)
152     p_values.append(p_val)
153
154 # 创建结果DataFrame
155 corr_df = pd.DataFrame({
156     'feature': X_scaled.columns,
157     'correlation': correlations,
158     'p_value': p_values,
159     'abs_correlation': np.abs(correlations)
160 }).sort_values('abs_correlation', ascending=False)
161
162 print("特征与目标变量的点二列相关系数:")
163 print(corr_df)
164
165 # 绘制相关系数图（改进版，与selectedCode风格一致）
166 fig, ax = plt.subplots(figsize=(12, 8))
167
168 y_pos = np.arange(len(corr_df))
169 bars = ax.barh(y_pos, corr_df['correlation'], color=colors[0], alpha=0.8)
170 ax.set_yticks(y_pos)
171 ax.set_yticklabels(corr_df['feature'])
172 ax.set_xlabel('点二列相关系数')
173 ax.set_title('特征与目标变量的点二列相关系数（标准化后）')
174 ax.grid(True, alpha=0.3)
175
176 # 在柱子上添加数值
177 for i, (bar, corr) in enumerate(zip(bars, corr_df['correlation'])):
178     ax.text(corr + (0.01 if corr >= 0 else -0.01), i, f'{corr:.3f}',
179             ha='left' if corr >= 0 else 'right', va='center', fontsize=9)
180
181 plt.tight_layout()
182 plt.savefig('point_biserial_correlations.png')
183 plt.close()
184
185 # 绘制相关系数绝对值的条形图
186 fig, ax = plt.subplots(figsize=(12, 8))
187 bars = ax.bar(range(len(corr_df)), corr_df['abs_correlation'],
188              color=colors[1], alpha=0.8)
189 ax.set_xlabel('特征')
190 ax.set_ylabel('点二列相关系数绝对值')
191 ax.set_title('特征与目标变量的点二列相关系数绝对值')
192 ax.set_xticks(range(len(corr_df)))
193 ax.set_xticklabels(corr_df['feature'], rotation=45, ha='right')
194 ax.grid(True, alpha=0.3)
195
196 # 在柱子上添加数值
197 for bar, abs_corr in zip(bars, corr_df['abs_correlation']):
198     height = bar.get_height()
199     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
200             f'{abs_corr:.3f}', ha='center', va='bottom', fontsize=9)
201
202 plt.tight_layout()
203 plt.savefig('point_biserial_correlations_abs.png')
204 plt.close()
205

```

```

206 # 根据阈值或top_k筛选特征
207 if top_k is not None:
208     selected_features = corr_df.head(top_k)['feature'].tolist()
209     print(f"\n选择前 {top_k} 个最相关的特征: {selected_features}")
210 else:
211     selected_features = corr_df[corr_df['abs_correlation'] >= threshold]['feature'].tolist()
212     print(f"\n选择相关系数绝对值 >= {threshold} 的特征: {selected_features}")
213
214 if not selected_features:
215     print("\n警告: 没有特征满足筛选条件, 使用所有特征")
216     return X
217
218 return X[selected_features] # 返回原始数据, 标准化只在筛选时使用
219
220 # 处理不平衡数据函数保持不变
221 def handle_imbalanced_data(X, y, method='smote'):
222     """处理不平衡数据"""
223     print(f"\n处理不平衡数据 - 方法: {method}")
224     print(f"原始数据分布: {Counter(y)}")
225
226     # 确保X是数值型
227     if not all(pd.api.types.is_numeric_dtype(X[col]) for col in X.columns):
228         print("错误: 特征矩阵包含非数值型数据")
229         return X, y
230
231     if method == 'smote':
232         sampler = SMOTE(random_state=42)
233     elif method == 'adasyn':
234         sampler = ADASYN(random_state=42)
235     elif method == 'smoteenn':
236         sampler = SMOTEENN(random_state=42)
237     else:
238         return X, y # 不处理
239
240     X_res, y_res = sampler.fit_resample(X, y)
241     print(f"处理后数据分布: {Counter(y_res)}")
242
243     return X_res, y_res
244
245 # 新增函数: 绘制交叉验证结果
246 def plot_cv_results(cv_results, model_name, scoring_metrics):
247     """绘制交叉验证结果"""
248     n_folds = len(cv_results['test_accuracy'])
249     x = np.arange(n_folds)
250     width = 0.15
251
252     fig, ax = plt.subplots(figsize=(12, 8))
253
254     for i, metric in enumerate(scoring_metrics):
255         scores = cv_results[f'test_{metric}']
256         ax.bar(x + i*width, scores, width, label=metric, alpha=0.8)
257
258     ax.set_xlabel('交叉验证折数')
259     ax.set_ylabel('分数')
260     ax.set_title(f'{model_name} 交叉验证结果')

```

```

261     ax.set_xticks(x + width*2)
262     ax.set_xticklabels([f'Fold {i+1}' for i in range(n_folds)])
263     ax.legend(loc='lower right')
264     ax.grid(True, alpha=0.3)
265
266     plt.tight_layout()
267     plt.savefig(f'{model_name}_cv_results.png')
268     plt.close()
269
270     # 绘制箱线图
271     fig, ax = plt.subplots(figsize=(10, 6))
272     scores_data = [cv_results[f'test_{metric}'] for metric in scoring_metrics]
273     ax.boxplot(scores_data, labels=scoring_metrics)
274     ax.set_title(f'{model_name} 交叉验证分数分布')
275     ax.set_ylabel('分数')
276     ax.grid(True, alpha=0.3)
277
278     plt.tight_layout()
279     plt.savefig(f'{model_name}_cv_boxplot.png')
280     plt.close()
281
282     # 新增函数：绘制ROC曲线比较
283     def plot_roc_curves(models, X_test, y_test, model_names):
284         """绘制ROC曲线比较"""
285         plt.figure(figsize=(10, 8))
286
287         for i, model in enumerate(models):
288             if hasattr(model, 'predict_proba'):
289                 y_prob = model.predict_proba(X_test)[: , 1]
290             else:
291                 y_prob = model.decision_function(X_test)
292
293             fpr, tpr, _ = roc_curve(y_test, y_prob)
294             auc_score = roc_auc_score(y_test, y_prob)
295
296             plt.plot(fpr, tpr, label=f'{model_names[i]} (AUC = {auc_score:.3f})', color=colors[i], linewidth=2)
297
298             plt.plot([0, 1], [0, 1], 'k--', label='随机猜测 (AUC = 0.5)')
299             plt.xlabel('假正率')
300             plt.ylabel('真正率')
301             plt.title('ROC曲线比较')
302             plt.legend(loc='lower right')
303             plt.grid(True, alpha=0.3)
304
305             plt.tight_layout()
306             plt.savefig('roc_curves_comparison.png')
307             plt.close()
308
309     # 新增函数：绘制精确率-召回率曲线比较
310     def plot_precision_recall_curves(models, X_test, y_test, model_names):
311         """绘制精确率-召回率曲线比较"""
312         plt.figure(figsize=(10, 8))
313
314         for i, model in enumerate(models):

```

```

315         if hasattr(model, 'predict_proba'):
316             y_prob = model.predict_proba(X_test)[:, 1]
317         else:
318             y_prob = model.decision_function(X_test)
319
320         precision, recall, _ = precision_recall_curve(y_test, y_prob)
321         avg_precision = np.mean(precision)
322
323         plt.plot(recall, precision, label=f'{model_names[i]} (AP = {avg_precision:.3f})', color=colors[i], linewidth=2)
324
325     plt.xlabel('召回率')
326     plt.ylabel('精确率')
327     plt.title('精确率-召回率曲线比较')
328     plt.legend(loc='upper right')
329     plt.grid(True, alpha=0.3)
330
331     plt.tight_layout()
332     plt.savefig('precision_recall_curves_comparison.png')
333     plt.close()
334
335 # 修改后的训练和评估函数
336 def train_and_evaluate_model(X, y, model_name='random_forest', use_cross_validation=True):
337     """训练和评估模型"""
338     # 检查数据是否有效
339     if X is None or y is None or len(X) == 0 or len(y) == 0:
340         print("错误: 无效的数据")
341         return None, 0, 0, 0, 0, {}
342
343     # 划分训练集和测试集
344     X_train, X_test, y_train, y_test = train_test_split(
345         X, y, test_size=0.3, random_state=42, stratify=y
346     )
347
348     # 选择模型
349     if model_name == 'random_forest':
350         model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
351     elif model_name == 'xgboost':
352         # 计算正负样本比例
353         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
354         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
355     elif model_name == 'svm':
356         model = SVC(random_state=42, class_weight='balanced', probability=True)
357     else:
358         model = RandomForestClassifier(random_state=42)
359
360     # 创建预处理管道
361     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
362     numeric_transformer = Pipeline(steps=[
363         ('scaler', StandardScaler())
364     ])
365
366     preprocessor = ColumnTransformer(
367         transformers=[
368             ('num', numeric_transformer, numeric_features)

```

```

369     ])
370
371     # 创建完整管道
372     clf = Pipeline(steps=[
373         ('preprocessor', preprocessor),
374         ('classifier', model)
375     ])
376
377     cv_results = {}
378     # 使用交叉验证
379     if use_cross_validation:
380         print(f"\n对 {model_name} 进行交叉验证...")
381
382         # 使用分层K折交叉验证
383         cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
384
385         # 定义评分指标
386         scoring_metrics = ['accuracy', 'f1', 'roc_auc', 'precision', 'recall']
387
388         # 执行交叉验证
389         for metric in scoring_metrics:
390             cv_scores = cross_val_score(clf, X, y, cv=cv, scoring=metric)
391             cv_results[f'test_{metric}'] = cv_scores
392             print(f"{model_name} 交叉验证 {metric}: {cv_scores.mean():.4f} (±{cv_scores.std():.4f})")
393
394         # 绘制交叉验证结果
395         plot_cv_results(cv_results, model_name, scoring_metrics)
396
397     # 训练最终模型
398     clf.fit(X_train, y_train)
399
400     # 预测
401     y_pred = clf.predict(X_test)
402     y_prob = clf.predict_proba(X_test)[:, 1]
403
404     # 评估模型
405     print(f"\n{model_name} 模型评估:")
406     print(classification_report(y_test, y_pred))
407
408     # 计算各种评估指标
409     accuracy = accuracy_score(y_test, y_pred)
410     precision = precision_score(y_test, y_pred)
411     recall = recall_score(y_test, y_pred)
412     auc_score = roc_auc_score(y_test, y_prob)
413     f1 = f1_score(y_test, y_pred)
414
415     print(f"准确率: {accuracy:.4f}")
416     print(f"精确率: {precision:.4f}")
417     print(f"召回率: {recall:.4f}")
418     print(f"AUC: {auc_score:.4f}")
419     print(f"F1分数: {f1:.4f}")
420
421     # 绘制混淆矩阵
422     cm = confusion_matrix(y_test, y_pred)
423     plt.figure(figsize=(6, 5))

```

```

424     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
425     plt.title(f'{model_name} 混淆矩阵')
426     plt.ylabel('真实标签')
427     plt.xlabel('预测标签')
428     plt.tight_layout()
429     plt.savefig(f'{model_name}_confusion_matrix.png')
430     plt.close()
431
432     # 绘制ROC曲线
433     fpr, tpr, _ = roc_curve(y_test, y_prob)
434     plt.figure(figsize=(8, 6))
435     plt.plot(fpr, tpr, color=colors[0], lw=2, label=f'ROC曲线 (AUC = {auc_score:.3f})')
436     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='随机猜测')
437     plt.xlim([0.0, 1.0])
438     plt.ylim([0.0, 1.05])
439     plt.xlabel('假正率')
440     plt.ylabel('真正率')
441     plt.title(f'{model_name} ROC曲线')
442     plt.legend(loc='lower right')
443     plt.grid(True, alpha=0.3)
444     plt.tight_layout()
445     plt.savefig(f'{model_name}_roc_curve.png')
446     plt.close()
447
448     # 绘制精确率-召回率曲线
449     precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_prob)
450     plt.figure(figsize=(8, 6))
451     plt.plot(recall_vals, precision_vals, color=colors[1], lw=2, label=f'精确率-召回率曲线')
452     plt.xlabel('召回率')
453     plt.ylabel('精确率')
454     plt.title(f'{model_name} 精确率-召回率曲线')
455     plt.legend(loc='upper right')
456     plt.grid(True, alpha=0.3)
457     plt.tight_layout()
458     plt.savefig(f'{model_name}_precision_recall_curve.png')
459     plt.close()
460
461     return clf, auc_score, f1, accuracy, recall, cv_results
462
463 # 新增函数：绘制模型指标比较图
464 def plot_metrics_comparison(metrics_df):
465     """绘制模型指标比较图"""
466     metrics = metrics_df.columns[1:] # 排除模型名列
467
468     # 计算需要的行数和列数
469     n_metrics = len(metrics)
470     n_cols = min(3, n_metrics) # 每行最多3列
471     n_rows = (n_metrics + n_cols - 1) // n_cols # 向上取整
472
473     fig, axes = plt.subplots(n_rows, n_cols, figsize=(5*n_cols, 5*n_rows))
474
475     # 如果只有一行或一列，确保axes是二维数组
476     if n_rows == 1 and n_cols == 1:
477         axes = np.array([[axes]])
478     elif n_rows == 1:

```

```

479     axes = axes.reshape(1, -1)
480 elif n_cols == 1:
481     axes = axes.reshape(-1, 1)
482
483 # 绘制每个指标的条形图
484 for i, metric in enumerate(metrics):
485     row = i // n_cols
486     col = i % n_cols
487     ax = axes[row, col]
488
489     bars = ax.bar(range(len(metrics_df)), metrics_df[metric],
490                   color=colors[:len(metrics_df)], alpha=0.8)
491     ax.set_xlabel('模型')
492     ax.set_ylabel(metric)
493     ax.set_title(f'{metric} 比较')
494     ax.set_xticks(range(len(metrics_df)))
495     ax.set_xticklabels(metrics_df['模型'], rotation=45)
496
497 # 在柱子上添加数值
498 for bar in bars:
499     height = bar.get_height()
500     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
501            f'{height:.3f}', ha='center', va='bottom', fontsize=9)
502
503     ax.grid(True, alpha=0.3)
504
505 # 隐藏多余的子图
506 for i in range(n_metrics, n_rows * n_cols):
507     row = i // n_cols
508     col = i % n_cols
509     axes[row, col].set_visible(False)
510
511 plt.tight_layout()
512 plt.savefig('metrics_comparison.png')
513 plt.close()
514
515 # 绘制雷达图
516 fig = plt.figure(figsize=(10, 10))
517 ax = fig.add_subplot(111, polar=True)
518
519 # 准备数据
520 categories = list(metrics)
521 N = len(categories)
522
523 # 计算每个模型的角度
524 angles = [n / float(N) * 2 * np.pi for n in range(N)]
525 angles += angles[:1] # 闭合雷达图
526
527 # 绘制每个模型的雷达图
528 for i, row in metrics_df.iterrows():
529     values = row[metrics].values.flatten().tolist()
530     values += values[:1] # 闭合雷达图
531     ax.plot(angles, values, linewidth=2, linestyle='solid',
532            label=row['模型'], color=colors[i])
533     ax.fill(angles, values, alpha=0.1, color=colors[i])

```



```

534
535 # 添加类别标签
536 plt.xticks(angles[:-1], categories)
537
538 # 添加图例
539 plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
540
541 # 添加标题
542 plt.title('模型性能雷达图比较', size=20, y=1.05)
543
544 plt.tight_layout()
545 plt.savefig('radar_chart_comparison.png')
546 plt.close()
547
548 # 修改后的比较采样方法函数
549 def compare_sampling_methods(X, y):
550     """比较不同的采样方法"""
551     methods = ['none', 'smote', 'smoteenn', 'adasyn']
552     results = []
553
554     for method in methods:
555         print(f"\n=== 使用方法: {method} ===")
556         X_res, y_res = handle_imbalanced_data(X, y, method)
557
558         # 使用随机森林评估每种方法
559         model, auc, f1, accuracy, recall, _ = train_and_evaluate_model(X_res, y_res, 'random_forest',
560                                 use_cross_validation=False)
561         results.append({
562             'method': method,
563             'auc': auc,
564             'f1': f1,
565             'accuracy': accuracy,
566             'recall': recall,
567             'distribution': Counter(y_res)
568         })
569
570 # 比较结果
571 results_df = pd.DataFrame(results)
572 print("\n=== 不同采样方法比较 ===")
573 print(results_df[['method', 'accuracy', 'recall', 'auc', 'f1']])
574
575 # 绘制比较图 (改进版, 与selectedCode风格一致)
576 fig, ax = plt.subplots(figsize=(12, 8))
577 x_pos = np.arange(len(results_df))
578 width = 0.35
579
580 bars1 = ax.bar(x_pos - width/2, results_df['auc'], width, label='AUC', color=colors[0], alpha=0.8)
581 bars2 = ax.bar(x_pos + width/2, results_df['f1'], width, label='F1 Score', color=colors[1], alpha
582               =0.8)
583
584 ax.set_xlabel('采样方法')
585 ax.set_ylabel('分数')
586 ax.set_title('不同采样方法性能比较')
587 ax.set_xticks(x_pos)
588 ax.set_xticklabels(results_df['method'])

```

```

587 ax.legend()
588 ax.grid(True, alpha=0.3)
589
590 # 在柱子上添加数值
591 for bar, auc_val in zip(bars1, results_df['auc']):
592     height = bar.get_height()
593     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
594             f'{auc_val:.3f}', ha='center', va='bottom', fontsize=9)
595
596 for bar, f1_val in zip(bars2, results_df['f1']):
597     height = bar.get_height()
598     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
599             f'{f1_val:.3f}', ha='center', va='bottom', fontsize=9)
600
601 plt.tight_layout()
602 plt.savefig('sampling_methods_comparison.png')
603 plt.close()
604
605 return results_df
606
607 # 超参数调优函数保持不变
608 def hyperparameter_tuning(X, y, model_name='random_forest'):
609     """超参数调优"""
610     print(f"\n=== 对 {model_name} 进行超参数调优 ===")
611
612     # 划分训练集和测试集
613     X_train, X_test, y_train, y_test = train_test_split(
614         X, y, test_size=0.3, random_state=42, stratify=y
615     )
616
617     # 创建预处理管道
618     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
619     numeric_transformer = Pipeline(steps=[
620         ('scaler', StandardScaler())
621     ])
622
623     preprocessor = ColumnTransformer(
624         transformers=[
625             ('num', numeric_transformer, numeric_features)
626         ])
627
628     # 根据模型选择不同的参数网格
629     if model_name == 'random_forest':
630         model = RandomForestClassifier(random_state=42, class_weight='balanced')
631         param_dist = {
632             'classifier__n_estimators': [50, 100, 200],
633             'classifier__max_depth': [None, 10, 20, 30],
634             'classifier__min_samples_split': [2, 5, 10],
635             'classifier__min_samples_leaf': [1, 2, 4]
636         }
637     elif model_name == 'xgboost':
638         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
639         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
640         param_dist = {
641             'classifier__n_estimators': [50, 100, 200],

```

```

642         'classifier__max_depth': [3, 6, 9],
643         'classifier__learning_rate': [0.01, 0.1, 0.2],
644         'classifier__subsample': [0.8, 0.9, 1.0]
645     }
646 elif model_name == 'svm':
647     model = SVC(random_state=42, class_weight='balanced', probability=True)
648     param_dist = {
649         'classifier__C': [0.1, 1, 10, 100],
650         'classifier__gamma': ['scale', 'auto', 0.01, 0.1, 1],
651         'classifier__kernel': ['rbf', 'linear']
652     }
653 else:
654     print(f"暂不支持 {model_name} 的超参数调优")
655     return None
656
657 # 创建完整管道
658 clf = Pipeline(steps=[
659     ('preprocessor', preprocessor),
660     ('classifier', model)
661 ])
662
663 # 使用随机搜索进行超参数调优
664 random_search = RandomizedSearchCV(
665     clf, param_distributions=param_dist, n_iter=20,
666     cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
667     scoring='f1', random_state=42, n_jobs=-1
668 )
669
670 # 执行随机搜索
671 random_search.fit(X_train, y_train)
672
673 # 输出最佳参数
674 print(f"最佳参数: {random_search.best_params_}")
675 print(f"最佳交叉验证分数: {random_search.best_score_:.4f}")
676
677 # 在测试集上评估最佳模型
678 best_model = random_search.best_estimator_
679 y_pred = best_model.predict(X_test)
680 y_prob = best_model.predict_proba(X_test)[: , 1]
681
682 # 计算评估指标
683 accuracy = accuracy_score(y_test, y_pred)
684 precision = precision_score(y_test, y_pred)
685 recall = recall_score(y_test, y_pred)
686 auc_score = roc_auc_score(y_test, y_prob)
687 f1 = f1_score(y_test, y_pred)
688
689 print(f"测试集准确率: {accuracy:.4f}")
690 print(f"测试集精确率: {precision:.4f}")
691 print(f"测试集召回率: {recall:.4f}")
692 print(f"测试集 AUC: {auc_score:.4f}")
693 print(f"测试集 F1 分数: {f1:.4f}")
694
695 # 绘制混淆矩阵
696 cm = confusion_matrix(y_test, y_pred)

```

```

697 plt.figure(figsize=(6, 5))
698 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
699 plt.title(f'{model_name} 超参数调优后混淆矩阵')
700 plt.ylabel('真实标签')
701 plt.xlabel('预测标签')
702 plt.tight_layout()
703 plt.savefig(f'{model_name}_tuned_confusion_matrix.png')
704 plt.close()
705
706 # 绘制ROC曲线
707 fpr, tpr, _ = roc_curve(y_test, y_prob)
708 plt.figure(figsize=(8, 6))
709 plt.plot(fpr, tpr, color=colors[0], lw=2, label=f'ROC曲线 (AUC = {auc_score:.3f})')
710 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='随机猜测')
711 plt.xlim([0.0, 1.0])
712 plt.ylim([0.0, 1.05])
713 plt.xlabel('假正率')
714 plt.ylabel('真正率')
715 plt.title(f'{model_name} 超参数调优后 ROC曲线')
716 plt.legend(loc='lower right')
717 plt.grid(True, alpha=0.3)
718 plt.tight_layout()
719 plt.savefig(f'{model_name}_tuned_roc_curve.png')
720 plt.close()
721
722 return best_model, {'accuracy': accuracy, 'precision': precision, 'recall': recall, 'auc': auc_score
723                     , 'f1': f1}
724
725 # 嵌入式特征选择函数保持不变
726 def embedded_feature_selection(X, y, model_name='random_forest', top_k=10):
727     """
728     使用嵌入式方法进行特征选择（基于树模型的特征重要性）
729     """
730     print(f"\n=== 使用 {model_name} 进行嵌入式特征选择 ===")
731
732     # 划分训练集和测试集
733     X_train, X_test, y_train, y_test = train_test_split(
734         X, y, test_size=0.3, random_state=42, stratify=y
735     )
736
737     # 选择模型
738     if model_name == 'random_forest':
739         model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
740     elif model_name == 'xgboost':
741         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
742         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
743     else:
744         print(f"暂不支持 {model_name} 的嵌入式特征选择")
745         return X
746
747     # 创建预处理管道
748     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
749     numeric_transformer = Pipeline(steps=[
750         ('scaler', StandardScaler())
751     ])

```

```

751
752 preprocessor = ColumnTransformer(
753     transformers=[
754         ('num', numeric_transformer, numeric_features)
755     ])
756
757 # 创建完整管道
758 clf = Pipeline(steps=[
759     ('preprocessor', preprocessor),
760     ('classifier', model)
761 ])
762
763 # 训练模型
764 clf.fit(X_train, y_train)
765
766 # 获取特征重要性
767 if hasattr(clf.named_steps['classifier'], 'feature_importances_'):
768     feature_importances = clf.named_steps['classifier'].feature_importances_
769     features = X.columns
770
771 # 创建特征重要性DataFrame
772 importance_df = pd.DataFrame({
773     'feature': features,
774     'importance': feature_importances
775 }).sort_values('importance', ascending=False)
776
777 print("特征重要性排名:")
778 print(importance_df)
779
780 # 绘制特征重要性图（改进版，与selectedCode风格一致）
781 fig, ax = plt.subplots(figsize=(12, 8))
782 y_pos = np.arange(len(importance_df.head(15)))
783 bars = ax.barh(y_pos, importance_df['importance'].head(15), color=colors[2], alpha=0.8)
784 ax.set_yticks(y_pos)
785 ax.set_yticklabels(importance_df['feature'].head(15))
786 ax.set_xlabel('重要性')
787 ax.set_title(f'{model_name} 特征重要性')
788 ax.grid(True, alpha=0.3)
789
790 # 在柱子上添加数值
791 for i, (bar, importance) in enumerate(zip(bars, importance_df['importance'].head(15))):
792     ax.text(importance + 0.001, i, f'{importance:.3f}',
793             ha='left', va='center', fontsize=9)
794
795 plt.tight_layout()
796 plt.savefig(f'{model_name}_feature_importance.png')
797 plt.close()
798
799 # 绘制特征重要性条形图
800 fig, ax = plt.subplots(figsize=(12, 8))
801 x_pos = np.arange(len(importance_df.head(15)))
802 bars = ax.bar(x_pos, importance_df['importance'].head(15), color=colors[3], alpha=0.8)
803 ax.set_xlabel('特征')
804 ax.set_ylabel('重要性')
805 ax.set_title(f'{model_name} 特征重要性条形图')

```

```

806     ax.set_xticks(x_pos)
807     ax.set_xticklabels(importance_df['feature'].head(15), rotation=45, ha='right')
808     ax.grid(True, alpha=0.3)
809
810     # 在柱子上添加数值
811     for bar, importance in zip(bars, importance_df['importance'].head(15)):
812         height = bar.get_height()
813         ax.text(bar.get_x() + bar.get_width()/2., height + 0.001,
814                 f'{importance:.3f}', ha='center', va='bottom', fontsize=9)
815
816     plt.tight_layout()
817     plt.savefig(f'{model_name}_feature_importance_bar.png')
818     plt.close()
819
820     # 选择前k个最重要的特征
821     selected_features = importance_df.head(top_k)['feature'].tolist()
822     print(f"\n选择前 {top_k} 个最重要的特征: {selected_features}")
823
824     return X[selected_features]
825 else:
826     print(f"{model_name} 模型不支持特征重要性分析")
827     return X
828
829 # 修改后的模型准确率和召回率比较图函数
830 def plot_model_metrics_comparison(model_results_df):
831     """绘制模型准确率和召回率比较图"""
832     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
833
834     # 准确率比较
835     bars1 = ax1.bar(range(len(model_results_df)), model_results_df['accuracy'],
836                     color=colors[0], alpha=0.8)
837     ax1.set_xlabel('模型')
838     ax1.set_ylabel('准确率')
839     ax1.set_title('不同模型准确率比较')
840     ax1.set_xticks(range(len(model_results_df)))
841     # 修改这里: 使用 '模型' 而不是 'model'
842     ax1.set_xticklabels(model_results_df['模型'])
843     ax1.grid(True, alpha=0.3)
844
845     # 在柱子上添加数值
846     for bar, acc_val in zip(bars1, model_results_df['accuracy']):
847         height = bar.get_height()
848         ax1.text(bar.get_x() + bar.get_width()/2., height + 0.01,
849                 f'{acc_val:.3f}', ha='center', va='bottom', fontsize=9)
850
851     # 召回率比较
852     bars2 = ax2.bar(range(len(model_results_df)), model_results_df['recall'],
853                     color=colors[1], alpha=0.8)
854     ax2.set_xlabel('模型')
855     ax2.set_ylabel('召回率')
856     ax2.set_title('不同模型召回率比较')
857     ax2.set_xticks(range(len(model_results_df)))
858     # 修改这里: 使用 '模型' 而不是 'model'
859     ax2.set_xticklabels(model_results_df['模型'])
860     ax2.grid(True, alpha=0.3)

```

```

861
862 # 在柱子上添加数值
863 for bar, rec_val in zip(bars2, model_results_df['recall']):
864     height = bar.get_height()
865     ax2.text(bar.get_x() + bar.get_width()/2., height + 0.01,
866             f'{rec_val:.3f}', ha='center', va='bottom', fontsize=9)
867
868 plt.tight_layout()
869 plt.savefig('model_accuracy_recall_comparison.png')
870 plt.close()
871
872 # 绘制准确率和召回率的组合图
873 fig, ax = plt.subplots(figsize=(10, 6))
874 x_pos = np.arange(len(model_results_df))
875 width = 0.35
876
877 bars1 = ax.bar(x_pos - width/2, model_results_df['accuracy'], width,
878               label='准确率', color=colors[0], alpha=0.8)
879 bars2 = ax.bar(x_pos + width/2, model_results_df['recall'], width,
880               label='召回率', color=colors[1], alpha=0.8)
881
882 ax.set_xlabel('模型')
883 ax.set_ylabel('分数')
884 ax.set_title('模型准确率和召回率比较')
885 ax.set_xticks(x_pos)
886 # 修改这里：使用 '模型' 而不是 'model'
887 ax.set_xticklabels(model_results_df['模型'])
888 ax.legend()
889 ax.grid(True, alpha=0.3)
890
891 # 在柱子上添加数值
892 for bar, acc_val in zip(bars1, model_results_df['accuracy']):
893     height = bar.get_height()
894     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
895            f'{acc_val:.3f}', ha='center', va='bottom', fontsize=9)
896
897 for bar, rec_val in zip(bars2, model_results_df['recall']):
898     height = bar.get_height()
899     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
900            f'{rec_val:.3f}', ha='center', va='bottom', fontsize=9)
901
902 plt.tight_layout()
903 plt.savefig('model_accuracy_recall_combined.png')
904 plt.close()
905
906 # 修改后的最终模型比较图函数
907 def plot_final_model_comparison(model_results_df):
908     """绘制最终模型比较图（与selectedCode风格一致）"""
909     metrics = ['auc', 'f1', 'accuracy', 'precision', 'recall']
910
911     fig, ax = plt.subplots(figsize=(12, 8))
912     x_pos = np.arange(len(model_results_df))
913     width = 0.15
914
915     for i, metric in enumerate(metrics):

```

```

916         ax.bar(x_pos + i*width, model_results_df[metric], width, label=metric, color=colors[i], alpha
917                =0.8)
918
919     ax.set_xlabel('模型')
920     ax.set_ylabel('分数')
921     ax.set_title('不同模型性能比较')
922     ax.set_xticks(x_pos + width*2)
923     # 修改这里: 使用 '模型' 而不是 'model'
924     ax.set_xticklabels(model_results_df['模型'])
925     ax.legend()
926     ax.grid(True, alpha=0.3)
927
928     plt.tight_layout()
929     plt.savefig('model_comparison.png')
930     plt.close()
931
932 # 修改后的主函数
933 def main():
934     """主函数"""
935     # 文件路径
936     input_file = "./python_code/Q4/处理后的女胎NIPT数据.xlsx"
937
938     # 加载和预处理数据
939     df = load_and_preprocess_data(input_file)
940     if df is None:
941         print("数据加载失败, 程序退出")
942         return
943
944     # 准备特征和目标
945     X, y = prepare_features(df)
946     if X is None or y is None:
947         print("特征准备失败, 程序退出")
948         return
949
950     # 检查是否有任何非数值型数据
951     print("\n检查数据类型...")
952     for col in X.columns:
953         if not pd.api.types.is_numeric_dtype(X[col]):
954             print(f"警告: 列 {col} 包含非数值型数据, 将被删除")
955             X = X.drop(columns=[col])
956
957     # 如果删除了列, 重新检查
958     if len(X.columns) == 0:
959         print("错误: 没有可用的数值型特征列")
960         return
961
962     print(f"初始特征: {list(X.columns)}")
963
964     # 第一步: 初步筛选 - 使用点二列相关系数
965     X_filtered = filter_features_point_biserial(X, y, threshold=0.05, top_k=15)
966
967     # 第二步: 精细筛选 - 使用嵌入式方法 (基于树模型的特征重要性)
968     # X_final = embedded_feature_selection(X_filtered, y, model_name='random_forest', top_k=10)
969     X_final = X_filtered

```



```

970     print(f"\n最终选择的特征: {list(X_final.columns)}")
971
972     # 比较不同的采样方法
973     results_df = compare_sampling_methods(X_final, y)
974
975     # 选择最佳方法
976     best_method = results_df.loc[results_df['f1'].idxmax(), 'method']
977     print(f"\n最佳采样方法: {best_method}")
978
979     # 应用最佳采样方法
980     X_res, y_res = handle_imbalanced_data(X_final, y, best_method)
981
982     # 训练不同模型并进行交叉验证
983     # 在 main() 函数中找到构建 model_results 的部分, 修改如下:
984
985     # 训练不同模型并进行交叉验证
986     models = ['random_forest', 'xgboost', 'svm']
987     trained_models = []
988     model_results = []
989     model_names = []
990
991     for model_name in models:
992         print(f"\n=== 训练 {model_name} 模型 ===")
993         model, auc, f1, accuracy, recall, cv_results = train_and_evaluate_model(X_res, y_res, model_name
            , use_cross_validation=True)
994
995         # 计算精确率
996         X_train, X_test, y_train, y_test = train_test_split(
997             X_res, y_res, test_size=0.3, random_state=42, stratify=y_res
998         )
999         y_pred = model.predict(X_test)
1000         precision = precision_score(y_test, y_pred)
1001
1002         model_results.append({
1003             '模型': model_name,
1004             'auc': auc,
1005             'f1': f1,
1006             'accuracy': accuracy,
1007             'precision': precision, # 这里应该是计算出的精度值, 而不是函数
1008             'recall': recall
1009         })
1010         trained_models.append(model)
1011         model_names.append(model_name)
1012
1013     # 创建模型指标DataFrame
1014     model_results_df = pd.DataFrame(model_results)
1015     print("\n=== 不同模型性能比较 ===")
1016     print(model_results_df)
1017
1018     # 绘制模型指标比较图
1019     plot_metrics_comparison(model_results_df)
1020
1021     # 绘制ROC曲线比较
1022     X_train, X_test, y_train, y_test = train_test_split(
1023         X_res, y_res, test_size=0.3, random_state=42, stratify=y_res

```

```

1024 )
1025 plot_roc_curves(trained_models, X_test, y_test, model_names)
1026
1027 # 绘制精确率-召回率曲线比较
1028 plot_precision_recall_curves(trained_models, X_test, y_test, model_names)
1029
1030 # 绘制模型比较图
1031 plot_final_model_comparison(model_results_df)
1032
1033 # 绘制准确率和召回率比较图
1034 plot_model_metrics_comparison(model_results_df)
1035
1036 # 选择最佳模型
1037 best_model_name = model_results_df.loc[model_results_df['f1'].idxmax(), '模型']
1038 print(f"\n最佳模型: {best_model_name}")
1039
1040 # 对最佳模型进行超参数调优
1041 best_model_tuned, tuned_metrics = hyperparameter_tuning(X_res, y_res, best_model_name)
1042
1043 # 重新训练最终模型
1044 print(f"\n=== 训练最终模型: {best_model_name} ===")
1045 final_model, final_auc, final_f1, final_accuracy, final_recall, _ = train_and_evaluate_model(X_res,
1046     y_res, best_model_name, use_cross_validation=True)
1047
1048 # 特征重要性 (如果适用)
1049 if hasattr(final_model.named_steps['classifier'], 'feature_importances_'):
1050     feature_importances = final_model.named_steps['classifier'].feature_importances_
1051     features = X_final.columns
1052
1053     # 创建特征重要性DataFrame
1054     importance_df = pd.DataFrame({
1055         'feature': features,
1056         'importance': feature_importances
1057     }).sort_values('importance', ascending=False)
1058
1059     print("\n最终模型特征重要性:")
1060     print(importance_df)
1061
1062     # 绘制特征重要性图 (改进版, 与selectedCode风格一致)
1063     fig, ax = plt.subplots(figsize=(12, 8))
1064     y_pos = np.arange(len(importance_df.head(15)))
1065     bars = ax.barh(y_pos, importance_df['importance'].head(15), color=colors[4], alpha=0.8)
1066     ax.set_yticks(y_pos)
1067     ax.set_yticklabels(importance_df['feature'].head(15))
1068     ax.set_xlabel('重要性')
1069     ax.set_title('最终模型 - Top 15 特征重要性')
1070     ax.grid(True, alpha=0.3)
1071
1072     # 在柱子上添加数值
1073     for i, (bar, importance) in enumerate(zip(bars, importance_df['importance'].head(15))):
1074         ax.text(importance + 0.001, i, f'{importance:.3f}',
1075             ha='left', va='center', fontsize=9)
1076
1077     plt.tight_layout()
1078     plt.savefig('final_feature_importance.png')

```

```

1078     plt.close()
1079
1080     # 绘制特征重要性条形图
1081     fig, ax = plt.subplots(figsize=(12, 8))
1082     x_pos = np.arange(len(importance_df.head(15)))
1083     bars = ax.bar(x_pos, importance_df['importance'].head(15), color=colors[5], alpha=0.8)
1084     ax.set_xlabel('特征')
1085     ax.set_ylabel('重要性')
1086     ax.set_title('最终模型 - 特征重要性条形图')
1087     ax.set_xticks(x_pos)
1088     ax.set_xticklabels(importance_df['feature'].head(15), rotation=45, ha='right')
1089     ax.grid(True, alpha=0.3)
1090
1091     # 在柱子上添加数值
1092     for bar, importance in zip(bars, importance_df['importance'].head(15)):
1093         height = bar.get_height()
1094         ax.text(bar.get_x() + bar.get_width()/2., height + 0.001,
1095                 f'{importance:.3f}', ha='center', va='bottom', fontsize=9)
1096
1097     plt.tight_layout()
1098     plt.savefig('final_feature_importance_bar.png')
1099     plt.close()
1100
1101     print("\n=== 分析完成 ===")
1102
1103 if __name__ == "__main__":
1104     main()

```